

Urządzenie plazmowe

Spis treści

1. Opis działania.....	2
1.1. Ekran główny - opis	2
1.2. Ekran nastawy parametrów - opis.....	3
1.3. Ekran diagnostyczny - opis	4
2. Spis urządzeń	4
3. Rysunki	5
3.1. Schemat montażowy elektryczny.....	5
3.2. Schemat złożeniowy techniczny	11
3.2.1. Podstawa	12
3.2.2. Stolik pod próbkę	17
3.2.3. Siłownik	21
3.2.4. Dolny moduł sterowania	23
3.2.5. Górny moduł sterowania.....	25
4. Kod programu.....	28
4.1 Wstępne uwagi.....	28
4.2 Skrypt wykonawczy	29
4.2.1. Konfiguracja.....	29
4.2.2. Baza danych.....	30
4.2.3. Jrk	31
4.2.4. DAC8532	32
4.2.5. ADS1256	32
4.2.6. usc02a.....	35
4.2.7. Wątki	35
4.2.8. Main.....	36
4.2.9. Bottle (web framework)	39
4.2.10. Część wykonawcza	44
4.2.11. Wyłączanie programu.....	45
4.3 Skrypty pomocnicze.....	46
4.3.1. SHT75 (sht71.py)	46
4.3.2. Tworzenie bazy danych (create_database.py)	46

1. Opis działania

Urządzenie pozwala na kontrolowane poddanie próbki wpływowi plazmy argonowej w celu przygotowania jej do pomiarów na urządzeniach zewnętrznych.

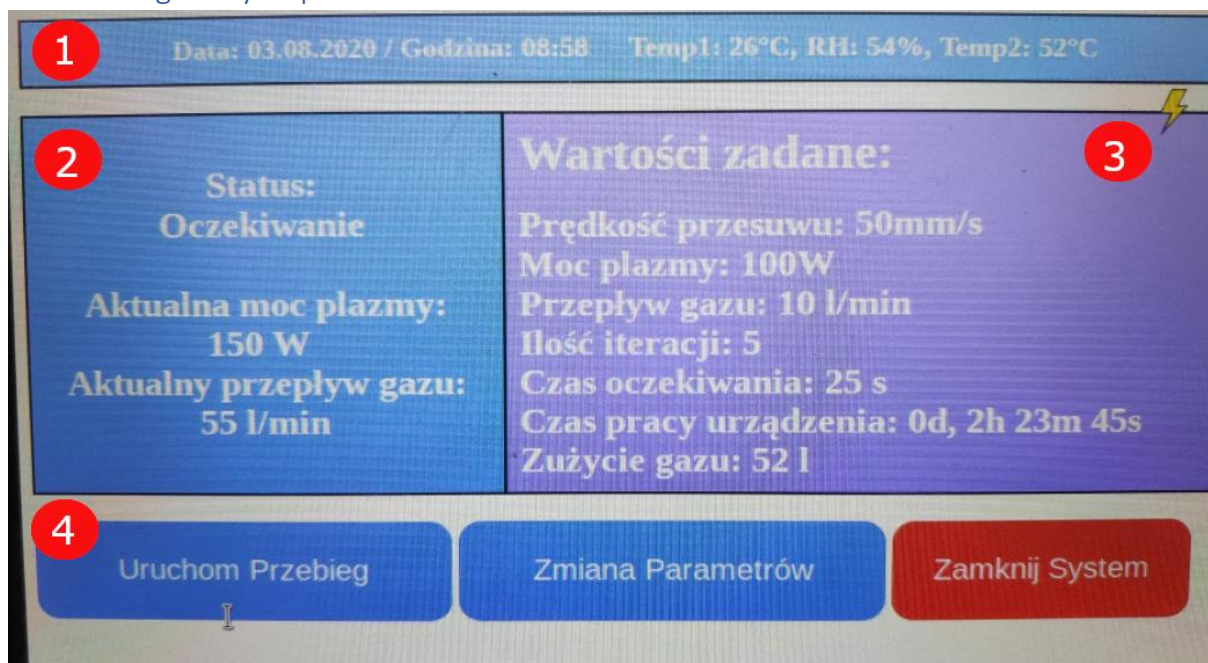
Sterowanie odbywa się za pomocą mikrokomputera Raspberry Pi 3. Na komunikację z użytkownikiem pozwala użycie dedykowanego ekranu dotykowego.

W trakcie uruchamiania systemu komputer uruchamia skrypt (/home/ielplasma/scripts/_www.py) oraz lokalną stronę internetową, odpowiedzialną za sterowanie pracą urządzenia od strony użytkownika.

Urządzenie samoczynnie ustawia tackę na próbkę na pozycji startowej, uruchamia zapłon oraz wczytuje parametry z lokalnej bazy danych.

Naciśnięcie przycisku „Uruchom Przebieg” rozpoczyna pracę urządzenia.

1.1. Ekran główny - opis

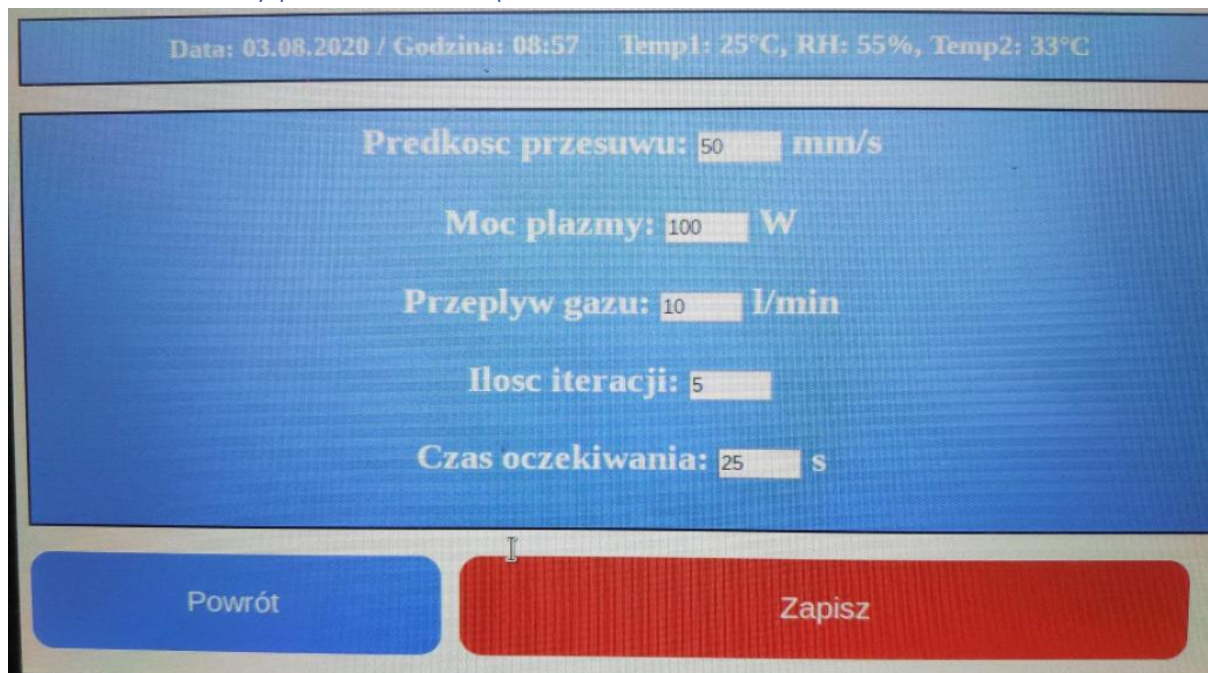


Rys.1.1. Ekran startowy aplikacji. Status, parametry i kontrola.

Opis poszczególnych bloków:

1. Data i godzina, odczyt z termometru DS18B20 oraz termohigrometru SHT75
2. Status urządzenia, aktualne parametry
3. Lista zadanych wartości
4. Kontrolki użytkownika:
 - a) Uruchomienie procesu
 - b) Przejście do ekranu zmiany parametrów
 - c) Zamknięcie systemu/Odczyt logów diagnostycznych

1.2. Ekran nastawy parametrów - opis



Rys.1.2. Dostosowywanie parametrów – widok

Zadane w widoku parametry przechowywane są w lokalnej bazie danych urządzenia. Po uruchomieniu programu zostają one wczytane. Jeżeli baza danych nie istnieje, jest ona tworzona na nowo. Parametry zadawane są w zakresie podanym poniżej w zestawieniu tabelarycznym.

Tab.1.1. Zakresy zadawanych parametrów

Lp.	Parametr	Minimalna wartość	Maksymalna wartość
1.	Prędkość przesuwu	7 mm/s	43 mm/s
2.	Moc plazmy	25 %	100 %
3.	Przepływ gazu	0.1 l/s	5 l/s
4.	Ilość iteracji	1	25
5.	Czas oczekiwania	100 ms	100 s

Opis parametrów:

- Prędkość przesuwu

Odpowiada za regulację prędkością przesuwu siłownika z zamontowaną tacką, na której umieszcza się próbkę;

- Moc plazmy

Tym parametrem regulujemy moc dostarczaną w celu zmiany stanu dostarczanego gazu;

- Przepływ gazu

Reguluje przepływ dostarczanego gazu;

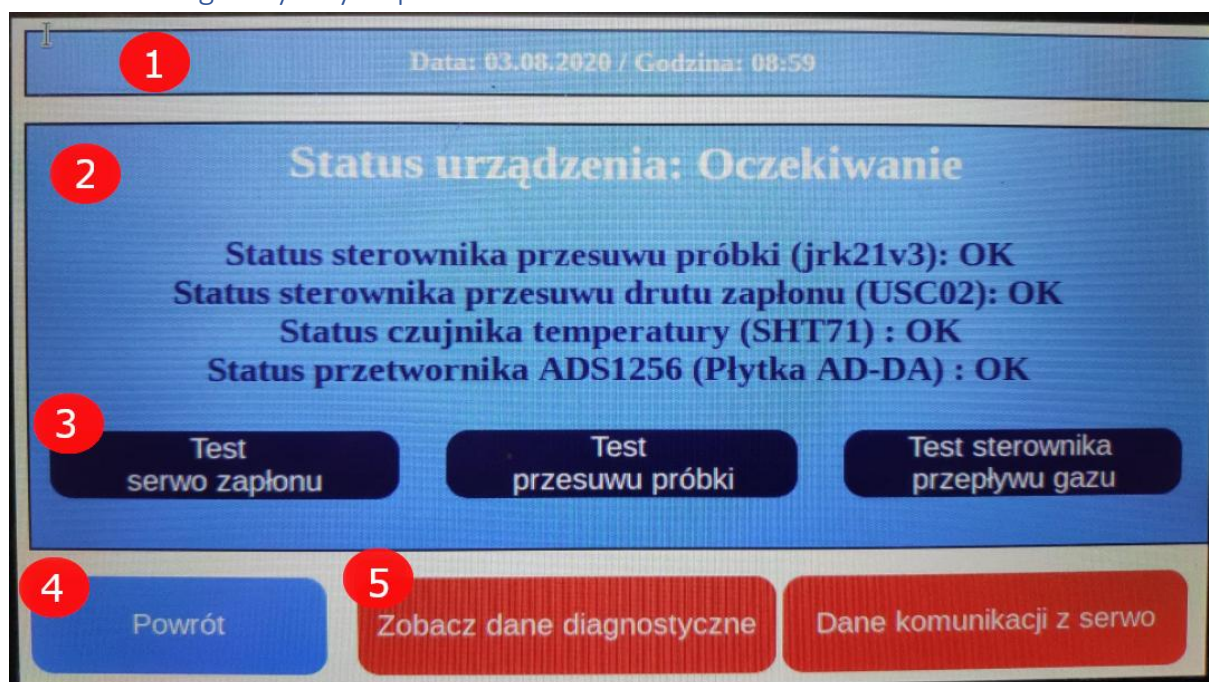
- Ilość iteracji

Tym parametrem ustalamy ilość cykli pracy urządzenia;

- Czas oczekiwania

Czas między kolejnymi cyklami procesu.

1.3. Ekran diagnostyczny - opis



Rys.1.3. Ekran diagnostyki

Opis poszczególnych bloków:

1. Aktualna data i godzina
2. Status urządzenia, czujników i sterowników
3. Przyciski przekierowujące do testów urządzeń:
 - a) Test serwomechanizmu
 - b) Test siłownika
 - c) Test sterownika przepływu gazu
4. Powrót do ekranu głównego
5. Logi diagnostyczne

2. Spis urządzeń

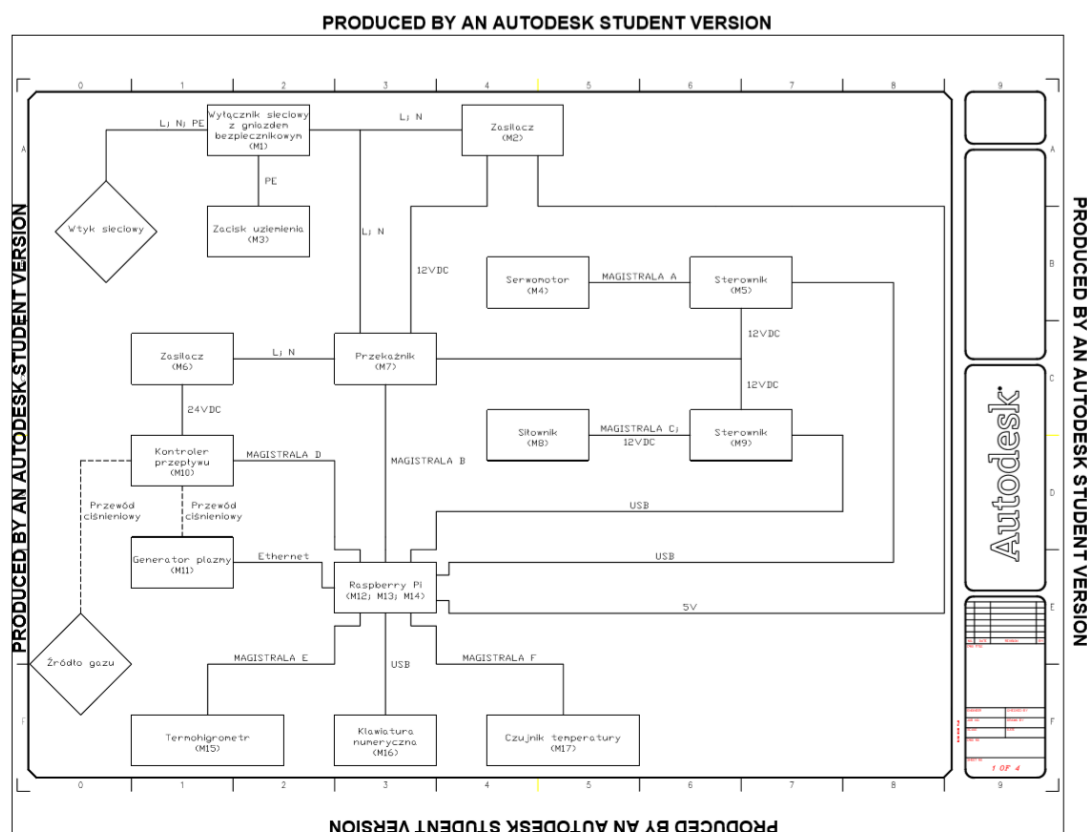
LP.	OZNACZENIE	URZĄDZENIE	MODEL	DOKUMENTACJA
1.	M1	Wyłącznik sieciowy z gniazdem bezpiecznikowym	DD11	https://www.schurter.cn/en/datasheet/typ_DD11.pdf
2.	M2	Zasilacz	PD-65	https://www.meanwell.com/Upload/PDF/PD-65/PD-65-SPEC.I
3.	M3	Zacisk uziemienia	-	-
4.	M4	Serwomotor	FS5106B	https://www.pololu.com/file/0J1432/FS5106B-specs.pdf
5.	M5	Sterownik serwomechanizmu	Micro Maestro 6	https://www.pololu.com/docs/0J40
6.	M6	Zasilacz	MFM-10-24	https://www.tme.eu/Document/dbd75bc9abb16a984d6083d

7.	M7	Przełącznik	4x SRD-05	https://datasheetspdf.com/datasheet/SRD-05VDC-SL-C.html
8.	M8	Siłownik	-	https://www.pololu.com/file/0J1238/LD-Linear-Actuator-Data
9.	M9	Sterownik siłownika	Polulu Jrk 21v3	https://www.pololu.com/docs/pdf/0J38/jrk_motor_controller
10.	M10	Kontroler przepływu	MFC D-5111	http://www.hoskinscientifique.com/uploadpdf/Instrumentation_stream_4b17d1e233ed6.pdf
11.	M11	Generator plazmy	-	-
12.	M12	Ekran dotykowy LCD	-	http://www.farnell.com/datasheets/1958036.pdf
13.	M13	Raspberry Pi	3	https://www.raspberrypi.org/documentation/hardware/comp
14.	M14	Przetwornik AC-CA	ADS1256 DAC8552	https://www.waveshare.com/w/upload/0/03/ADS1256.pdf https://www.waveshare.com/w/upload/1/1b/DAC8552.pdf
15.	M15	Termohigrometr	SHT-75	https://www.mouser.com/datasheet/2/682/Sensirion_Humid
16.	M16	Klawiatura numeryczna	NKL-1333	https://www.01.ee/en/product/1365202
17.	M17	Czujnik temperatury	DS18B20	http://www.farnell.com/datasheets/2345098.pdf?_ga=2.7512346036407.1592297449&_gac=1.226474472.1596446921.Cj0ksMB6ceHTwKcidlrsMP9z9V6ls3r1kvqKsgrDVHd8e8RjDlpipPlp

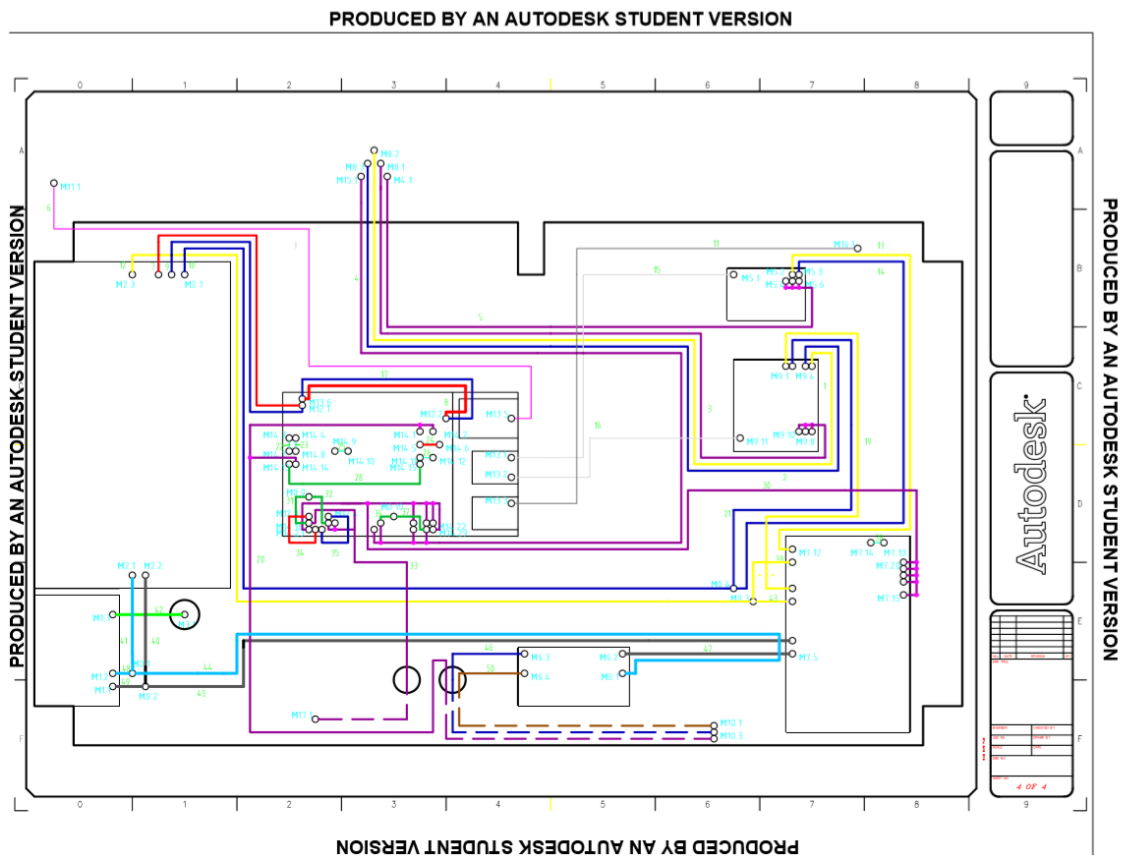
3. Rysunki

3.1. Schemat montażowy elektryczny

[Trawiarka plazmowa. Schemat montażowy elektryczny\SCHEMAT MONTAŻOWY.pdf](#)



Rys.3.1.1. Schemat blokowy urządzenia



Rys.3.1.2. Schemat połączeń

OZNACZENIE	PIN	PRZEWODY		MODUŁ
M0.1		41	ŻÓŁTY 0.75mm ²	ZŁĄCZKA
		44	NIEBIESKI 1.5mm ²	
		48	NIEBIESKI 1.5mm ²	
M0.2		40	SZARY 0.75mm ²	ZŁĄCZKA
		45	BRAZOWY 1.5mm ²	
		49	BRAZOWY 1.5mm ²	
M0.3		12	ŁOSOSIOWY 0.75mm ²	ZŁĄCZKA
		39	ŁOSOSIOWY 0.75mm ²	
		43	ŁOSOSIOWY 0.75mm ²	
M0.4		10	CZARNY 0.75mm ²	ZŁĄCZKA
		14	CZARNY 0.75mm ²	
		21	CZARNY 0.75mm ²	
M0.9		31	SYGNAŁOWY	REZYSTOR 10kOhm
		32	SYGNAŁOWY	
M0.10		36	SYGNAŁOWY	REZYSTOR 10kOhm
		37	SYGNAŁOWY	
M1.1	L	48	NIEBIESKI 1.5mm ²	DD11.0113.1111
M1.2	N	49	BRAZOWY 1.5mm ²	
M1.3	PE	42	CZERWONY 0.75mm ²	
M2.1	AC/N	41	ŻÓŁTY 0.75mm ²	PD-65
M2.2	AC/L	40	SZARY 0.75mm ²	
M2.3	+V	12	ŁOSOSIOWY 0.75mm ²	
M2.5	+5V	7	CZARNY 0.5mm ²	
M2.6	COM	9	SZMARAGDOWY 0.5mm ²	
M2.7	COM	10	CZARNY 0.5mm ²	
M3.1		42	CZERWONY 0.75mm ²	OBUDOWA
M4.1		5	MAGISTRALA A	FS5106B
M5.1	MINI B USB CONNECTOR	15	USB	MICRO MAESTRO 6
M5.2	+ SERVO POWER	13	ŁOSOSIOWY 0.75mm ²	
M5.3	-SERVO POWER	14	CZARNY 0.75mm ²	
M5.4				
M5.5		5	MAGISTRALA A	
M5.6				

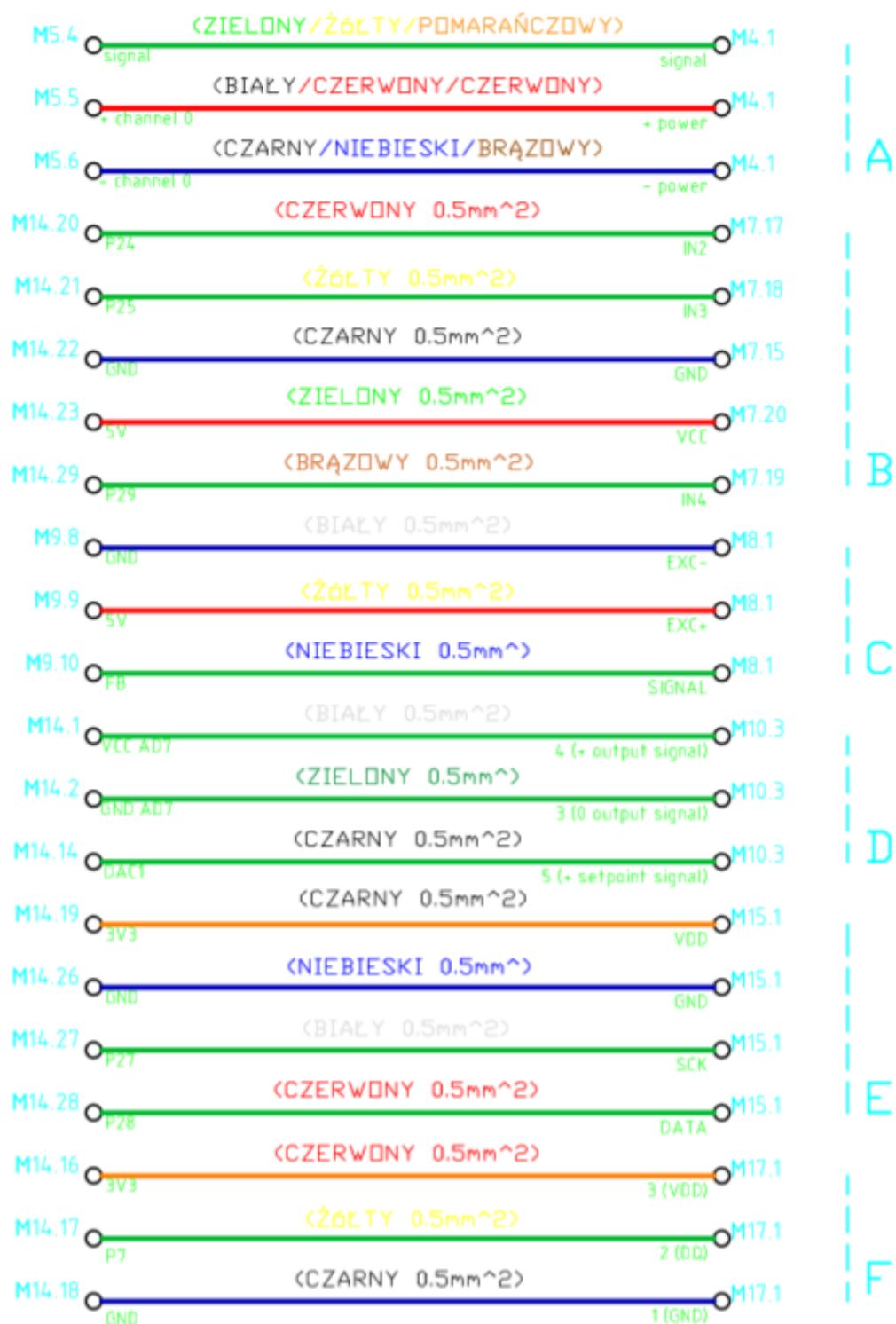
Rys.3.1.3a. Opis połączeń 1/4

OZNACZENIE	PIN	PRZEWODY		MODUŁ
M6.1	ACN	44	NIEBIESKI 1.5mm^2	MFM-10-24
M6.2	ACL	47	BRAZOWY 1.5mm^2	
M6.3	0V POWER	46	CZERWONY 0.75mm^2	
M6.4	+ POWER	50	CZARNY 0.75mm^2	
M7.5	K2.COM	47	BRAZOWY 1.5mm^2	4 CHANNEL RELAY MODULE
M7.6	K2.NO	45	BRAZOWY 1,5mm^2	
M7.8	K3.COM	43	ŁOSOSIOWY 0.75mm^2	
M7.9	K3.NO	19	ŁOSOSIOWY 0.75mm^2	
M7.11	K4.COM	39	ŁOSOSIOWY 0.75mm^2	
M7.12	K4.NO	13	ŁOSOSIOWY 0.75mm^2	
M7.13	VCC	38	ZWORA	
M7.14	JD-VCC	38		
M7.15		30	MAGISTRALA B	
M7.16				
M7.17				
M7.18				
M7.19				
M7.20				
M8.1		3	MAGISTRALA C	GLIDEFORCE LIGHT DUTY LINEAR ACTUATOR
M8.2	+POWER	1	CZARNY 0.5mm^2	
M8.3	-POWER	2	CZERWONY 0.5mm^2	
M9.1	VIN	19	ŁOSOSIOWY 0.75mm^2	POLDLU JRK 21v3 USB MOTOR CONTROLLER WITH FEEDBACK
M9.2	GND	21	CZARNY 0.75mm^2	
M9.3	MOTOR OUTPUT A	1	CZARNY 0.5mm^2	
M9.4	MOTOR OUTPUT B	2	CZERWONY 0.5mm^2	
M9.8		3	MAGISTRALA C	
M9.9				
M9.10				
M9.11	MINI B USB CONNECTOR	16	microUSB	MASS FLOW CONTROLLER D-5111
M10.1	+POWER	50	CZARNY 0.75mm^2	
M10.2	0POWER	46	CZERWONY 0.75mm^2	
M10.3		20	MAGISTRALA D	GENERATOR PLAZMY
M11.1	ETHERNET PORT	6	ETHERNET	
M12.1	PWR IN	7	CZARNY 0.75mm^2	RASPBERRY PI 7 TOUCHSCREEN DISPLAY - RASPBERRY PI DISPLAY V1.1 (2015)
		9	SZMARAGDOWY 0.5mm^2	
M12.2	PWR OUT	17	CZARNY 0.75mm^2	
		8	CZERWONY 0.75mm^2	
M12.3	5V	34	CZERWONY 0.5mm^2	
M12.4	GND	35	CZARNY 0.5mm^2	

Rys.3.1.3b. Opis połączeń 2/4

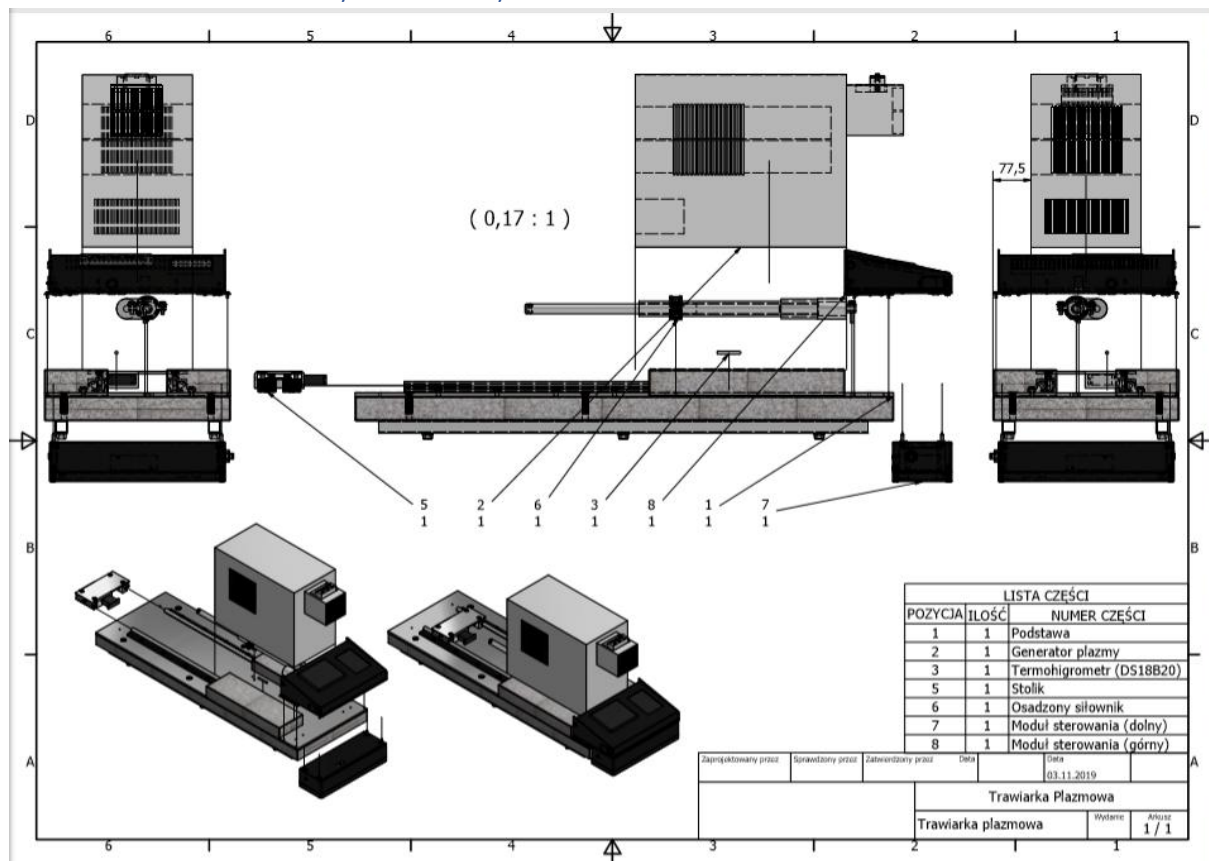
OZNACZENIE	PIN	PRZEWODY		MODUŁ	
M13.1	J11 (DOLNY)	15	USB	RASPBERRY PI 7 TOUCHSCREEN DISPLAY – RASPBERRY PI 3 MODEL B V1.2 (2015)	
M13.2	J11 (GÓRNY)	16	USB		
M13.3	J12 (GÓRNY)	11	USB		
M13.5	J10	6	ETHERNET		
M13.6	PWR IN	17	CZARNY 0.75mm^2		
		8	CZERWONY 0.75mm^2		
M14.1		20	MAGISTRALA D	RASPBERRY PI HIGH-PRECISSION AD/DA EXPANSION BOARD	
M14.2					
M14.3	5V	22	ZWORA A		
M14.4	5V	23	ZWORA B		
M14.5	AD5	24	CZERWONY 0.5_mm^2		
M14.6	VCC AD5				
M14.7	VREF	22	ZWORA A		
M14.8	VCC	23	ZWORA B		
M14.9	AGND	25	ZWORA C		
M14.10	AINCOM				
M14.11	AD3	26	ZWORA D		
M14.12	GND AD3				
M14.13	DAC0	28	BIAŁY 0.5MM^2		
M14.14		20	MAGISTRALA D		
M14.15	AD2	28	BIAŁY 0.5MM^2		
M14.16	3V3	31	SYGNAŁOWY		
		33	MAGISTRALA F		
M14.17	P7	32	SYGNAŁOWY		
		33	MAGISTRALA F		
M14.18	GND	4	MAGISTRALA E		
M14.19	3V3	36	SYGNAŁOWY		
M14.20		30	MAGISTRALA B		
M14.21					
M14.22					
M14.23					
M14.24	5V	34	CZERWONY 0.5_mm^2		
M14.25	GND	35	CZARNY 0.5mm^2		
M14.26			MAGISTRALA E		
M14.27					
M14.28					
M14.28	P28	37	SYGNAŁOWY		
M14.29		30	MAGISTRALA B		
M15.1		4	MAGISTRALA E	SHT75	
M16.1		11	USB	NKL-1333	
M17.1		33	MAGISTRALA F	DS18B20	

Rys.3.1.3c. Opis połączeń 3/4



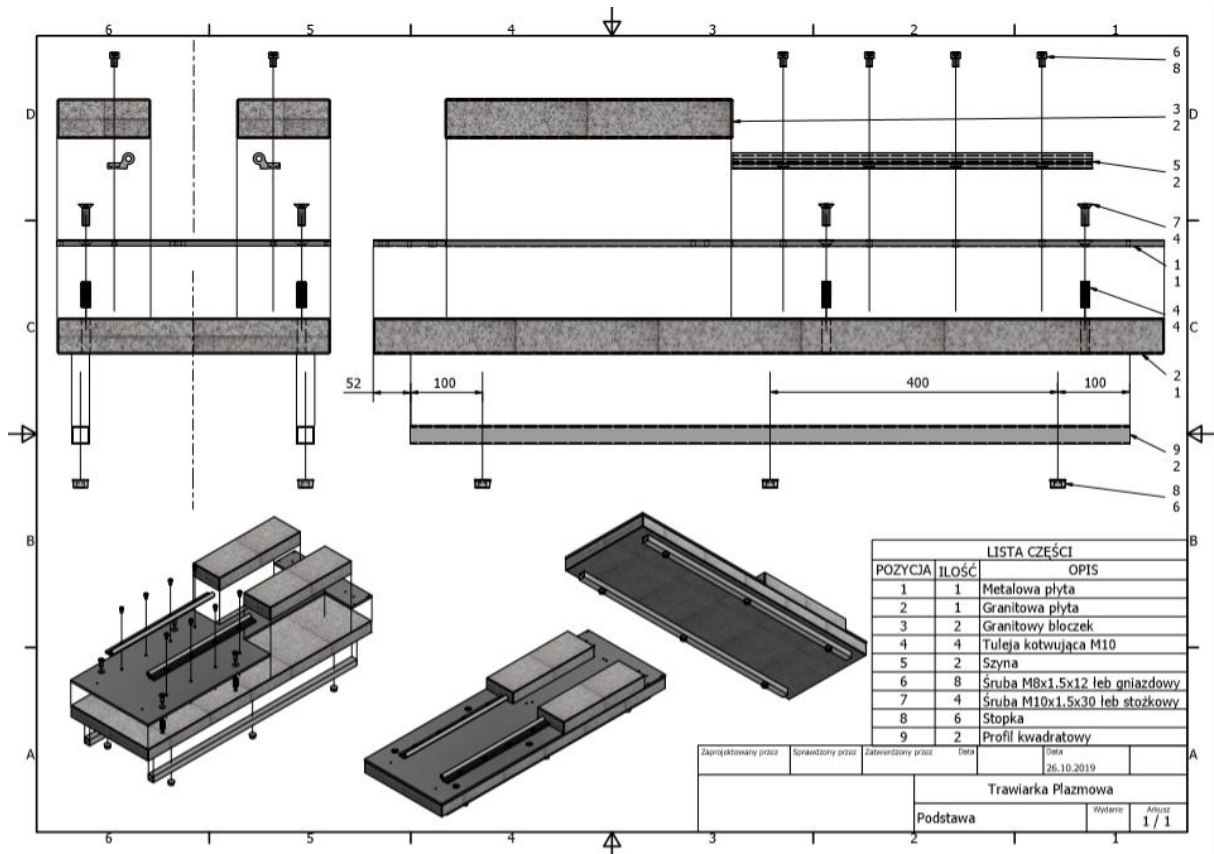
Rys.3.1.3d. Opis połączeń 4/4 - magistrale

3.2. Schemat złożeniowy techniczny

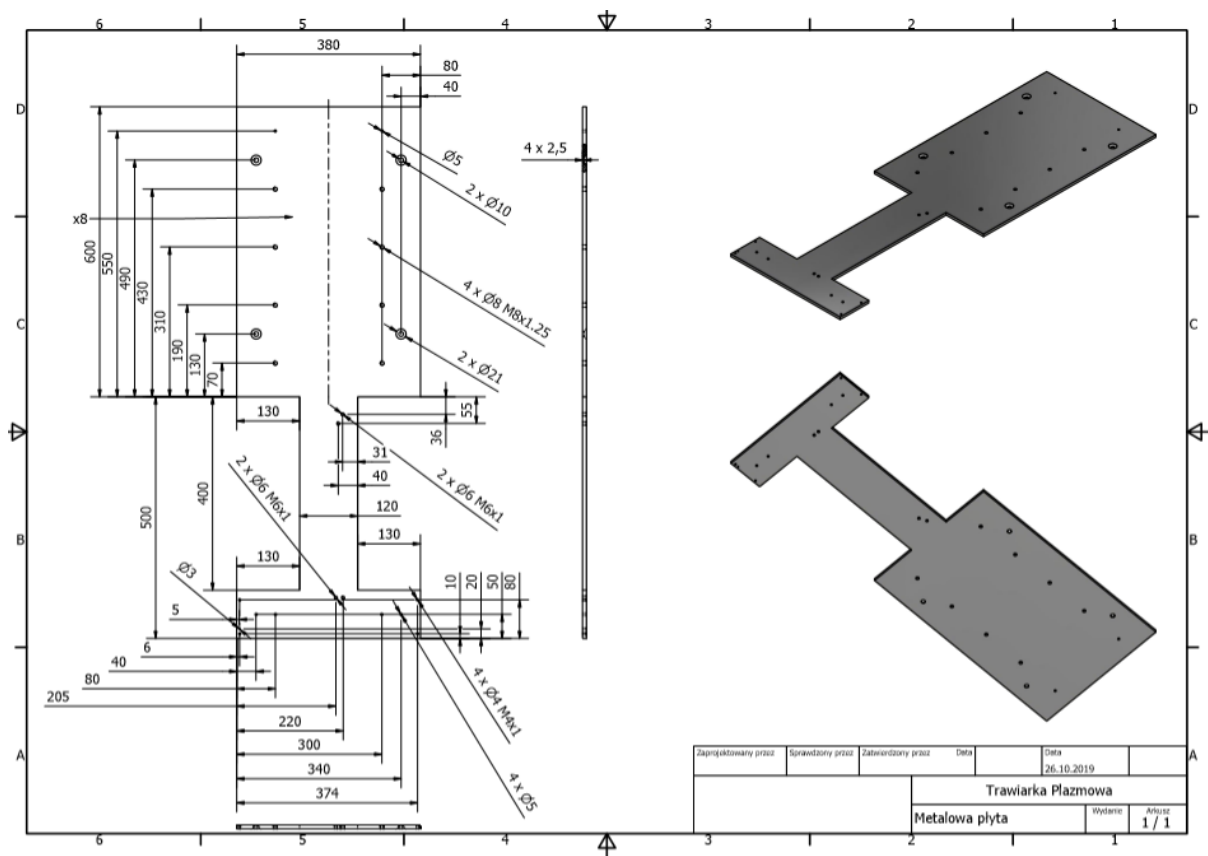


Rys.3.2.1 Urządzenie plazmowe – rysunek złożeniowy ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.0.0 Trawiarka plazmowa.pdf](#))

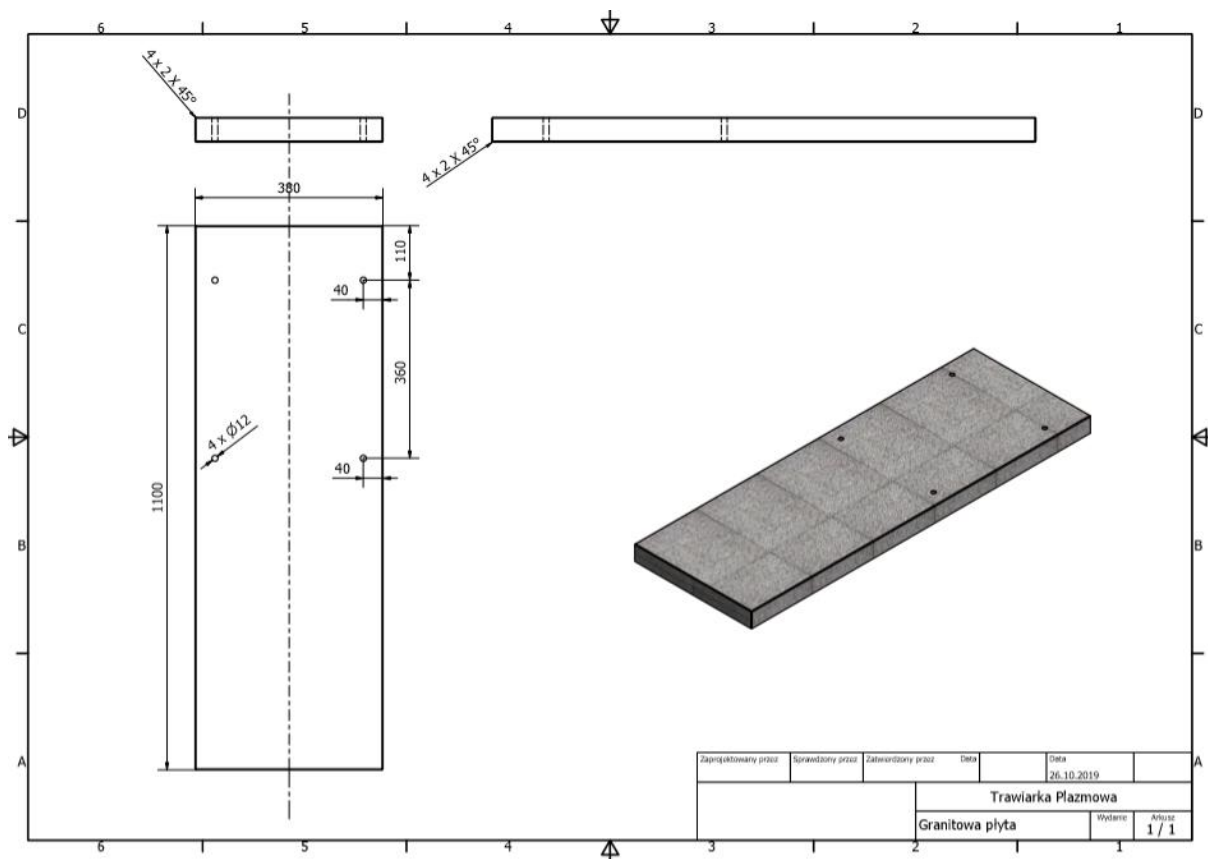
3.2.1. Podstawa



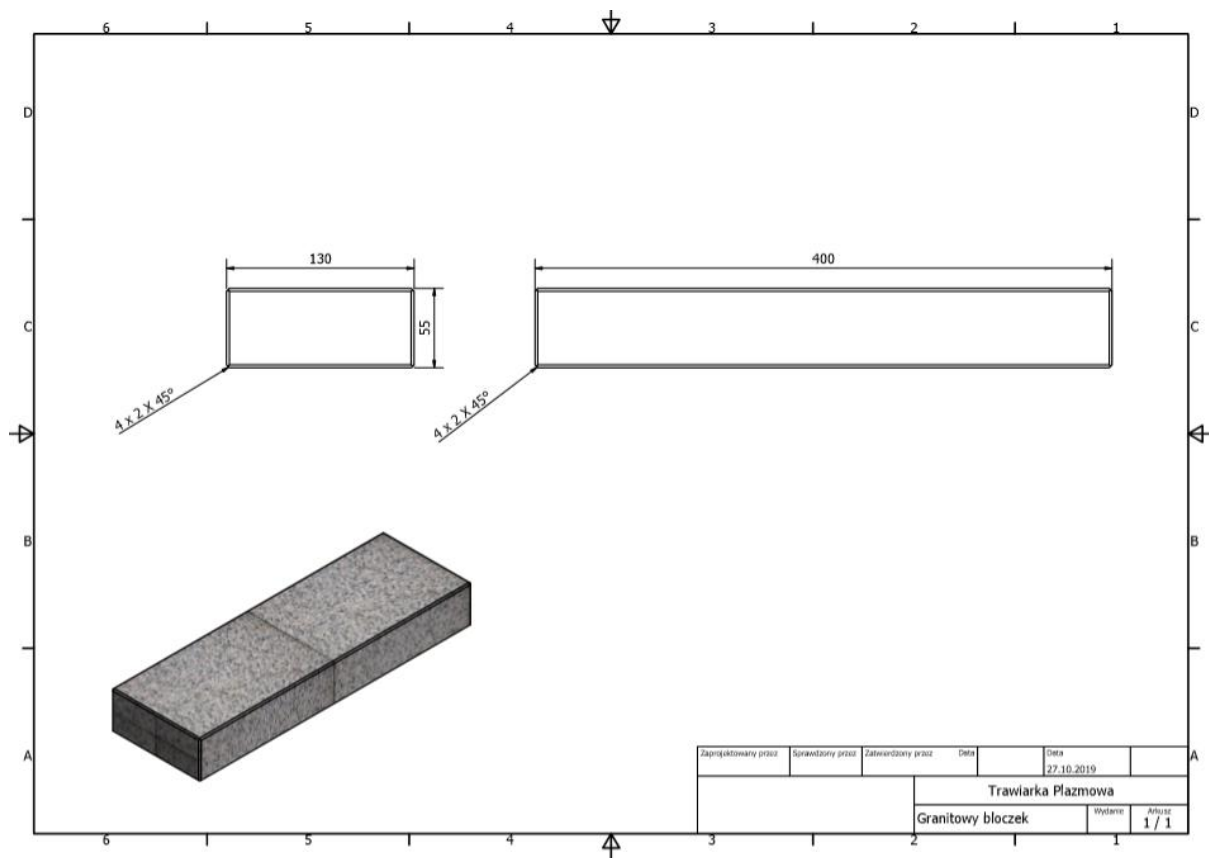
Rys.3.2.2.1. Podstawa – rysunek złożeniowy ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.1.0 Podstawa.pdf](#))



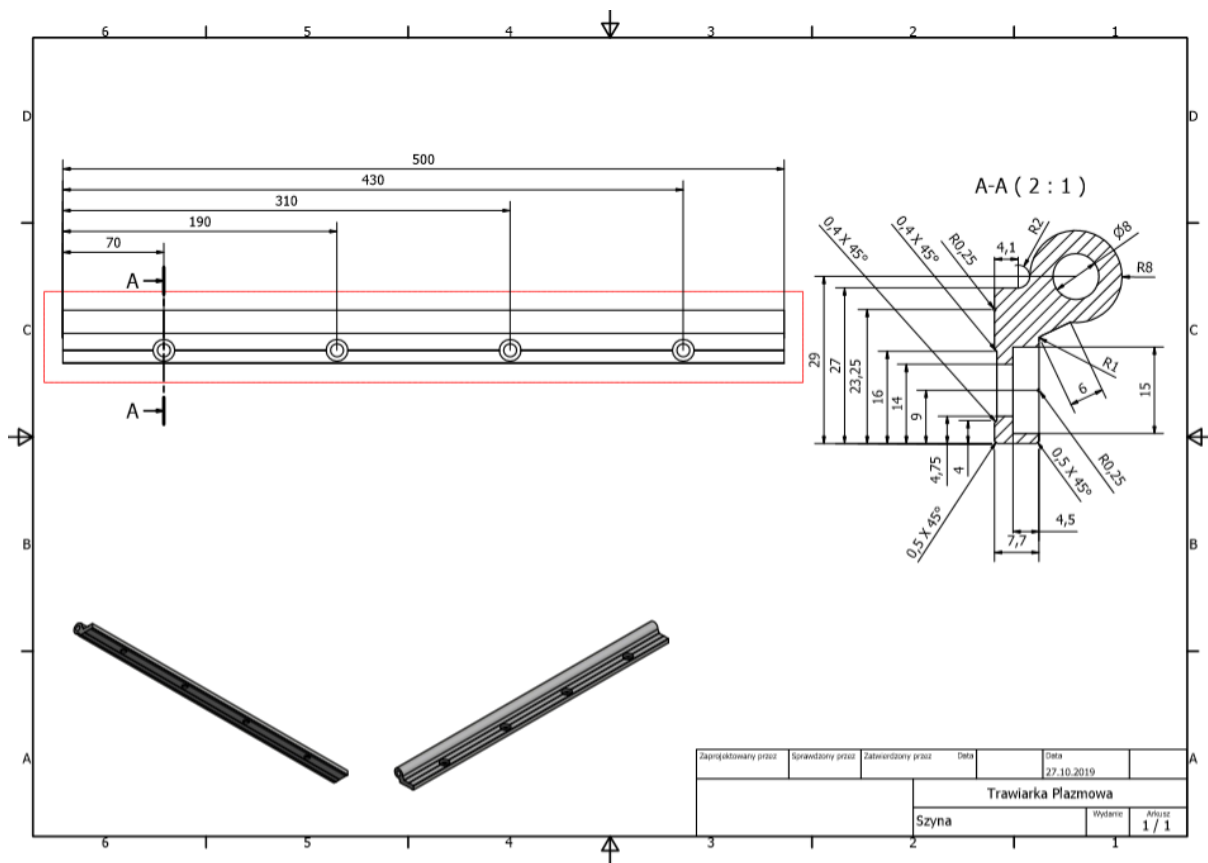
Rys.3.2.2.2. Metalowa płyta ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.1.1 Metalowa płyta.pdf](#))



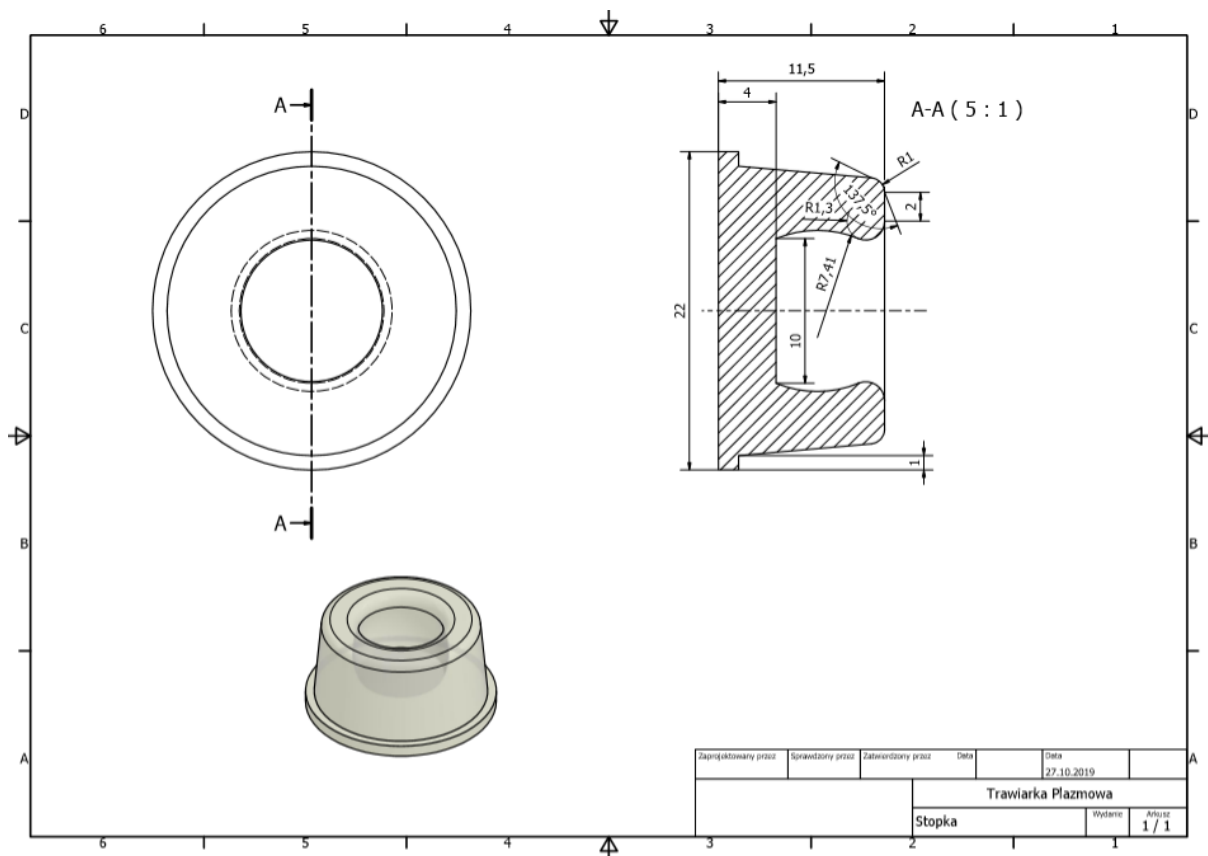
Rys.3.2.2.3. Granitowa płyta ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.1.2 Granitowa płyta.pdf](#))



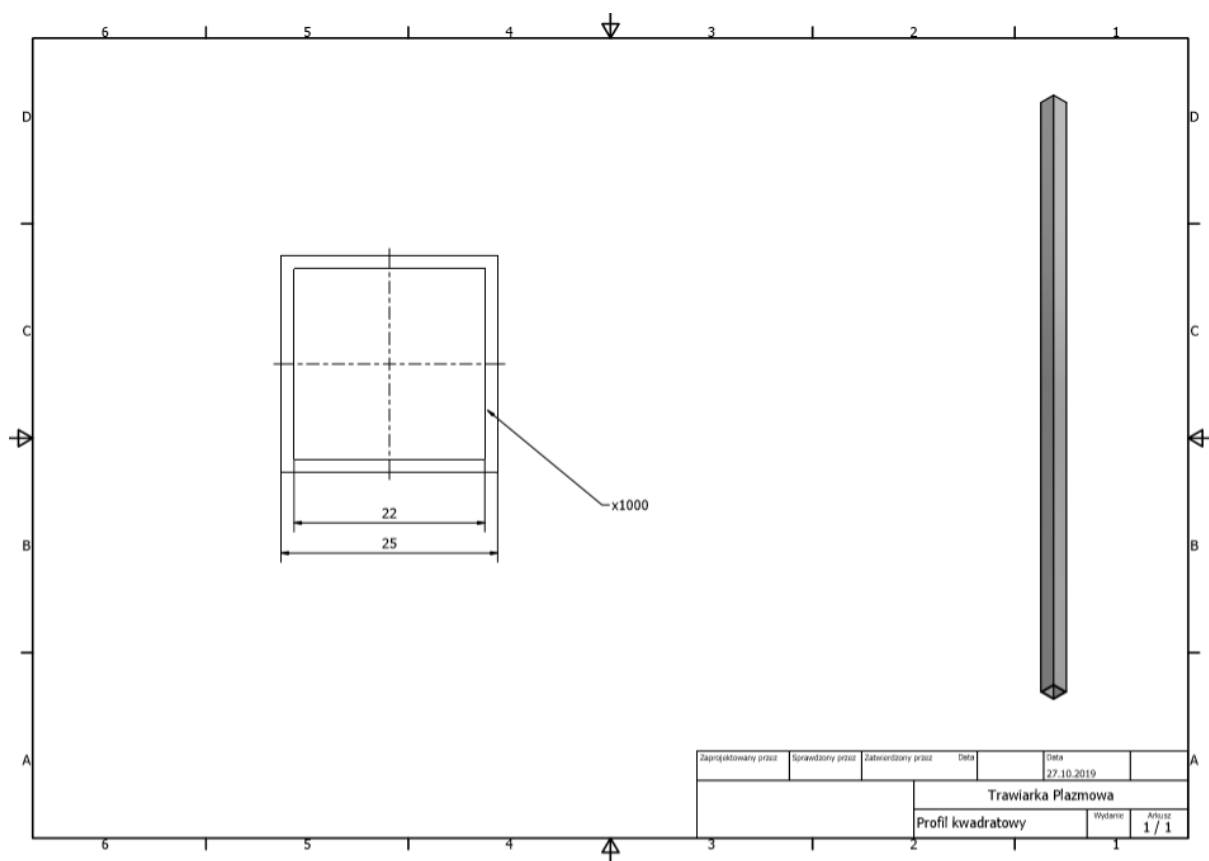
Rys.3.2.2.4. Granitowy bloczek ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.1.3 Granitowy bloczek.pdf](#))



Rys.3.2.2.5. Szyrna ([Trawiarka Plazowa. Schemat złozeniowy techniczny\1.1.5 Szyrna.pdf](#))

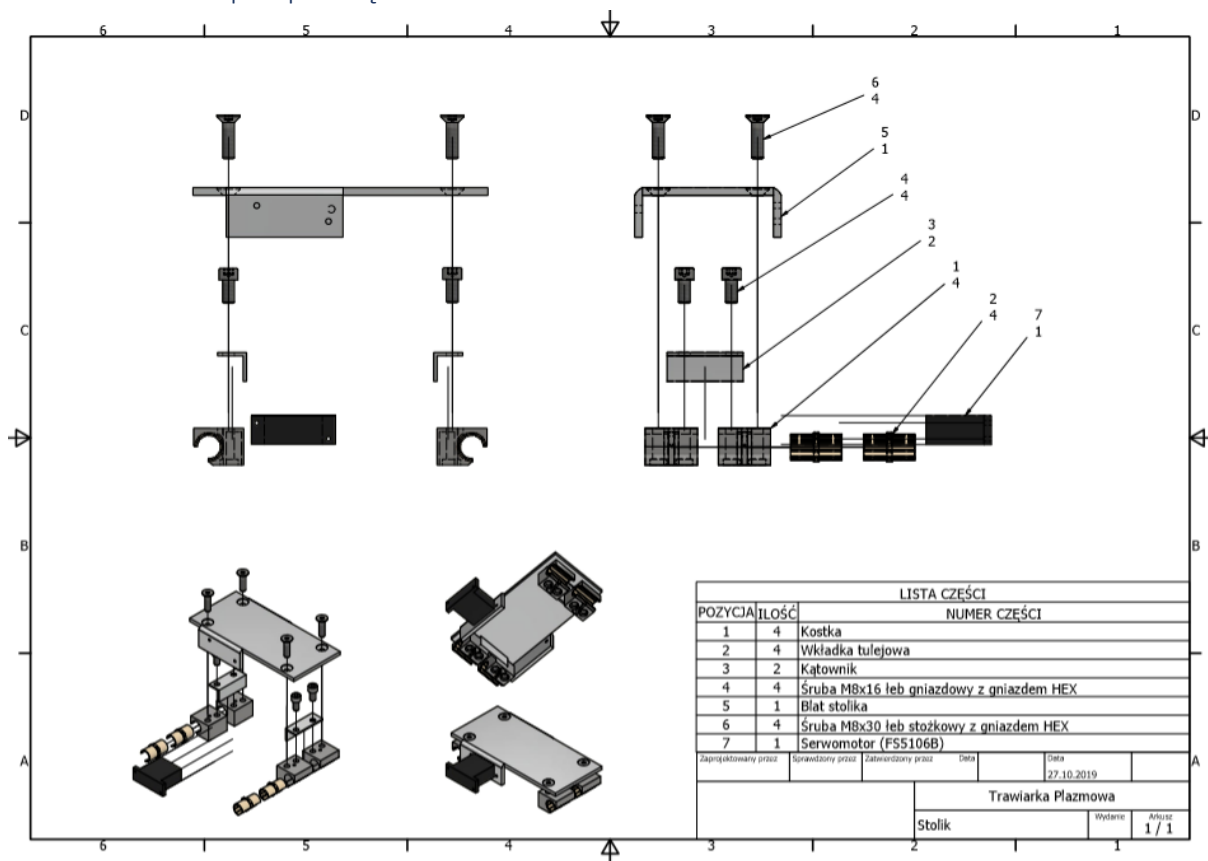


Rys.3.2.2.6. Stopka ([Trawiarka Plazowa. Schemat złozeniowy techniczny\1.1.8 Stopka.pdf](#))

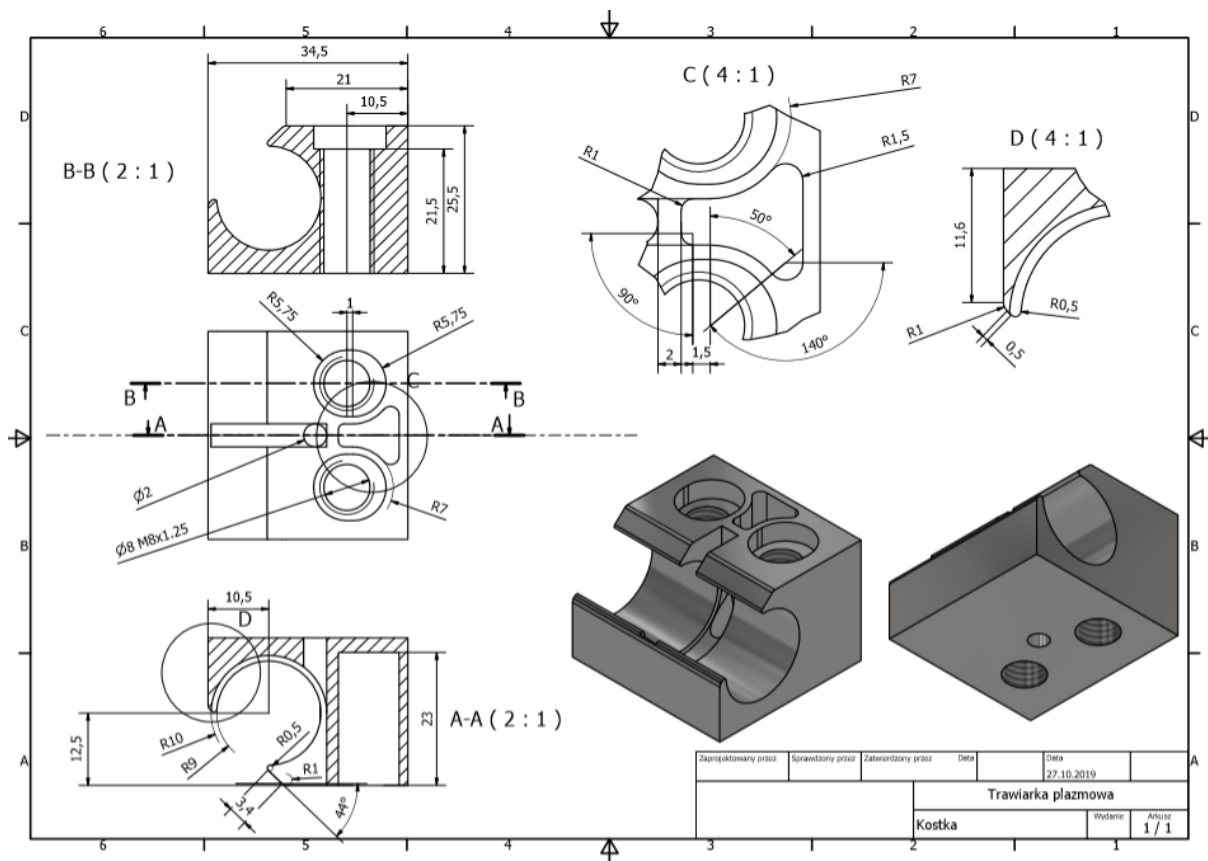


Rys.3.2.2.7. Profil kwadratowy ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.1.9 Profil kwadratowy.pdf](#))

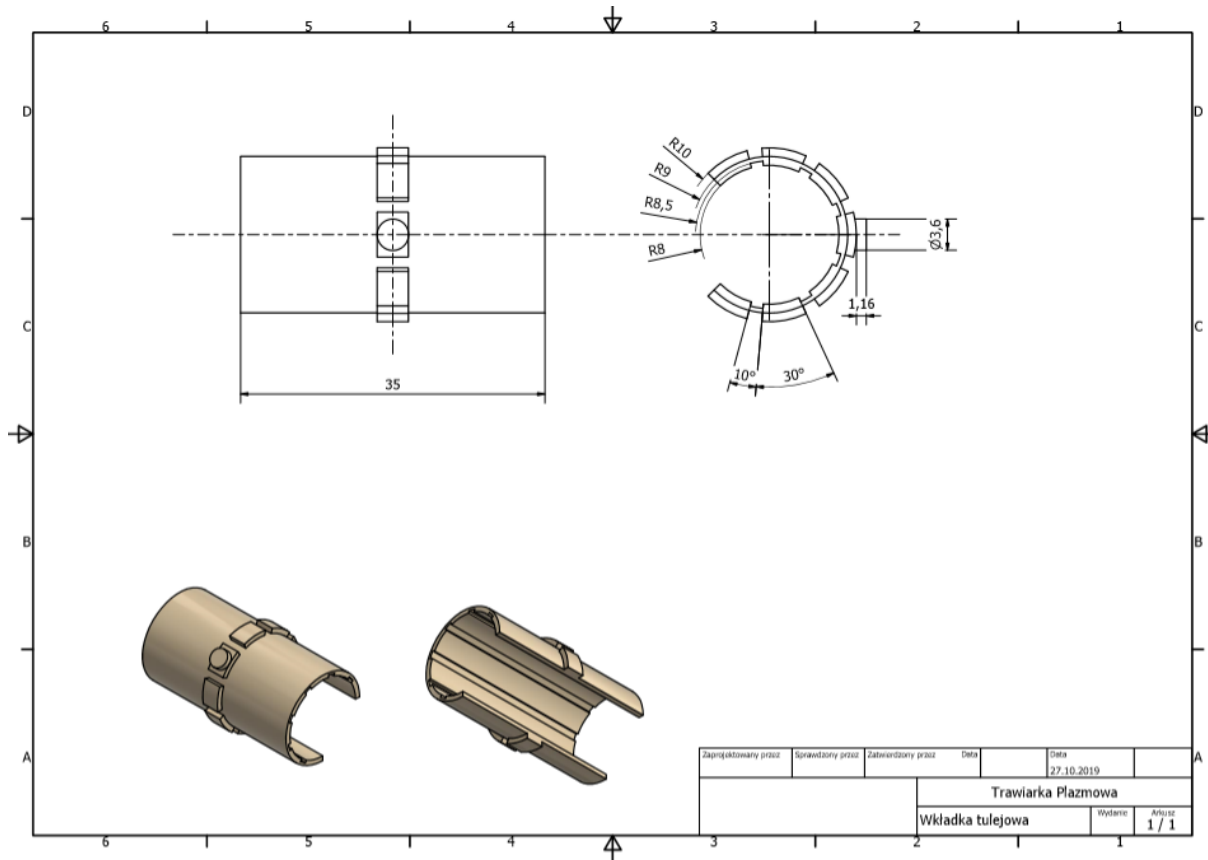
3.2.2. Stolik pod próbkę



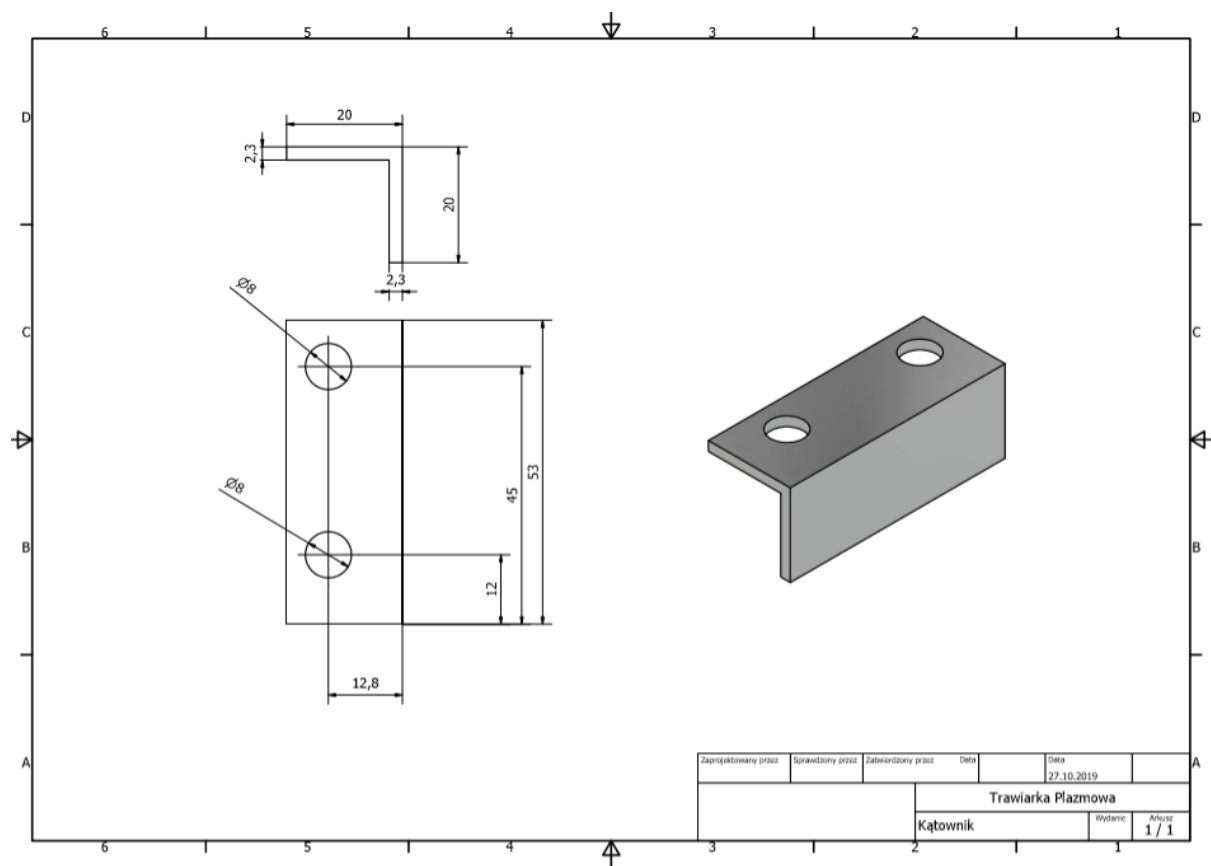
Rys.3.2.3.1. Stolik pod próbkę – rysunek złożeniowy ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.5.0 Stolik.pdf](#))



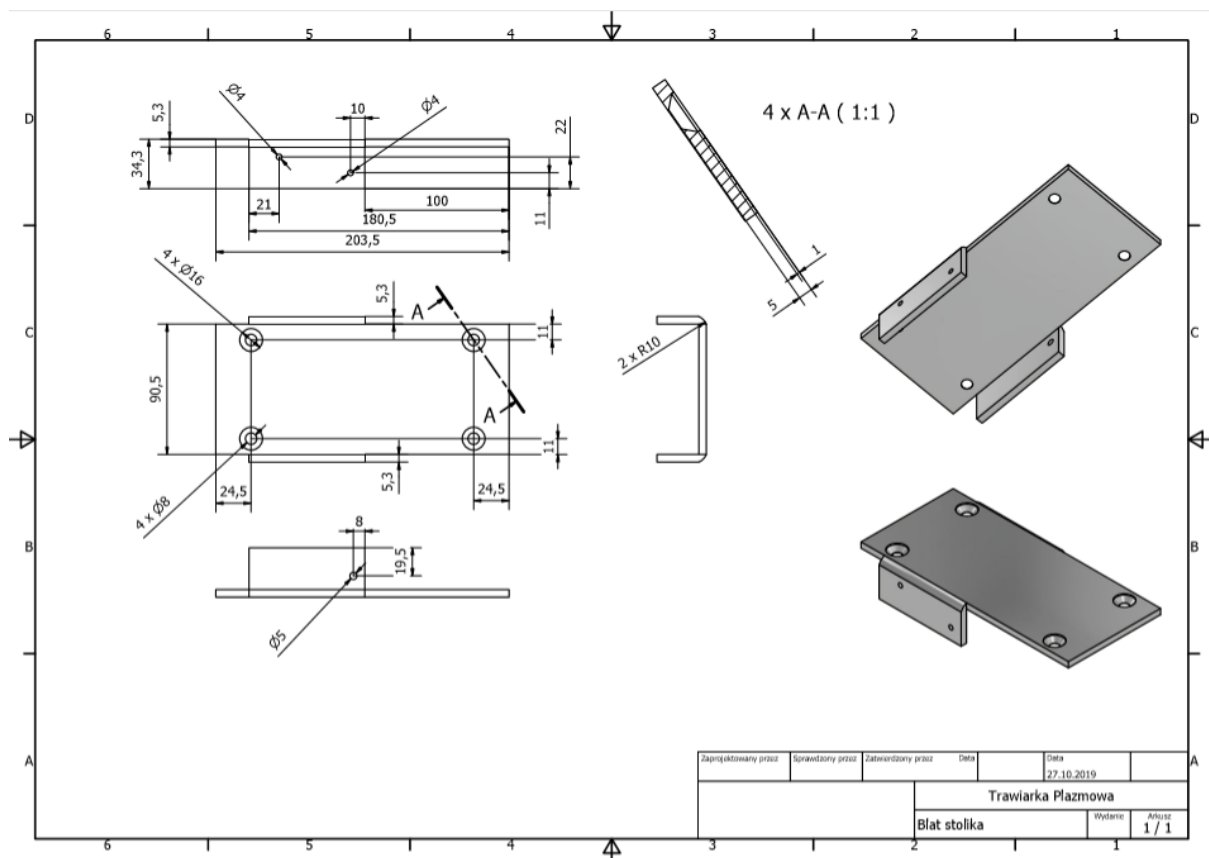
Rys.3.2.3.2. Kostka ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.5.1 Kostka.pdf](#))



Rys.3.2.3.3. Wkładka tulejowa ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.5.2 Wkładka tulejowa.pdf](#))

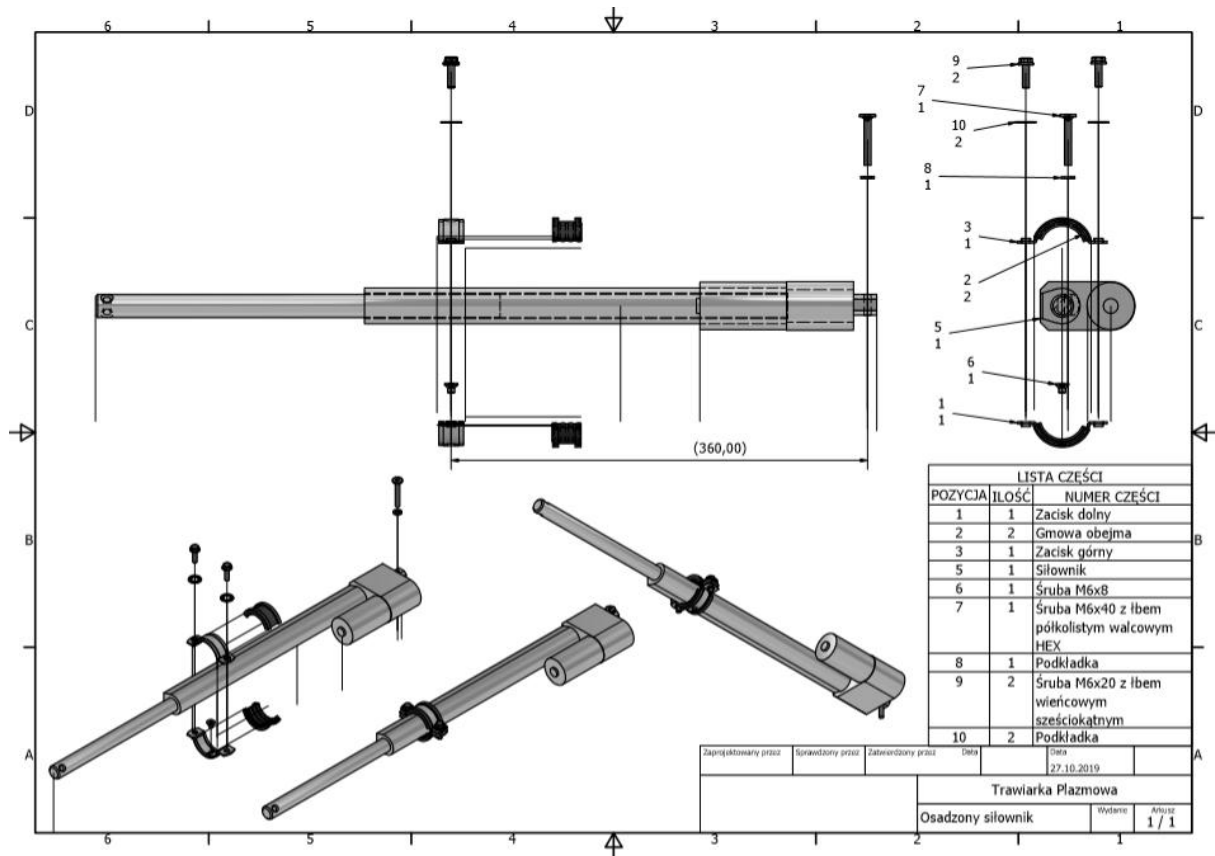


Rys.3.2.3.4 Kątownik ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.5.3 Kątownik.pdf](#))

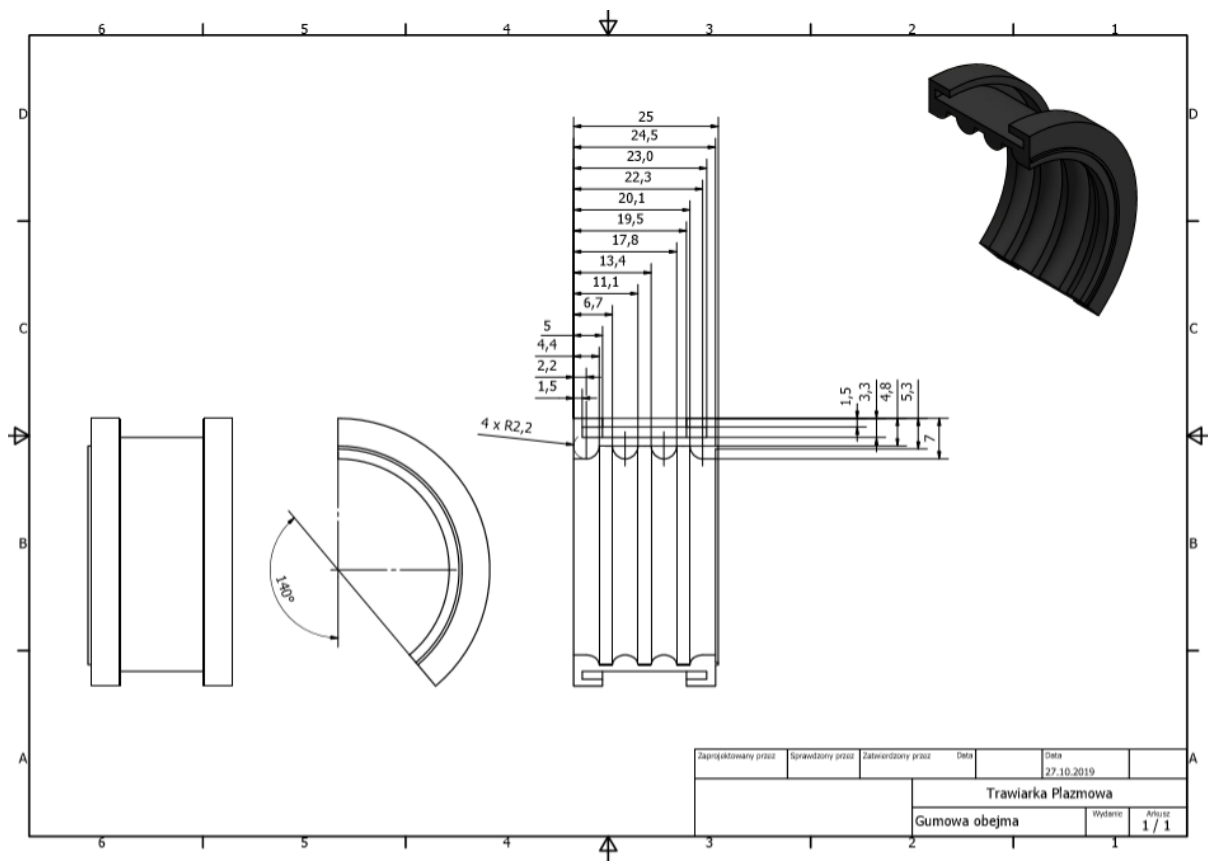


Rys.3.2.3.5. Błat stolika ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.5.5 Błat stolika.pdf](#))

3.2.3. Siłownik

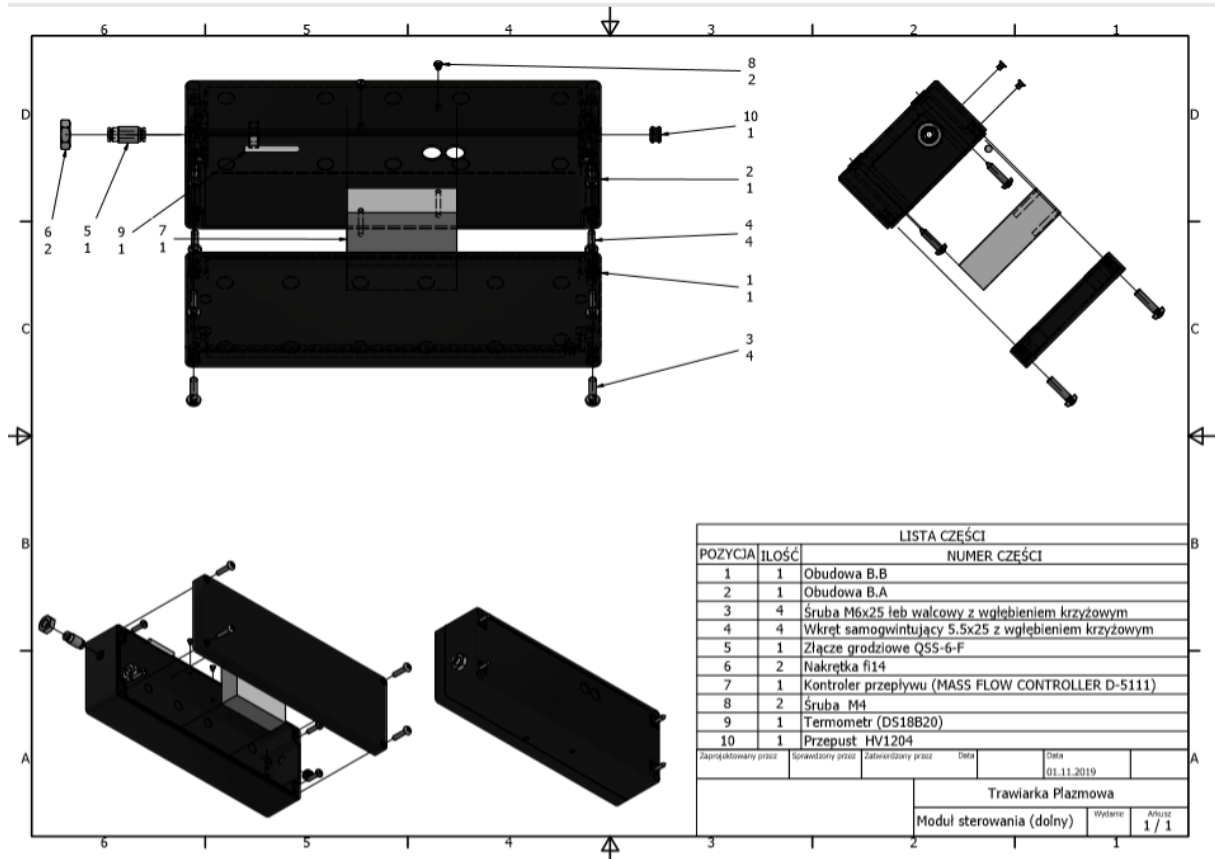


Rys.3.2.4.1. Siłownik – schemat złożeniowy ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.6.5 Siłownik.pdf](#))



Rys.3.2.4.2. Gumowa obejma ([Trawiarka plazmowa. Schemat złożeniowy techniczny\1.6.2 Gumowa obejma.pdf](#))

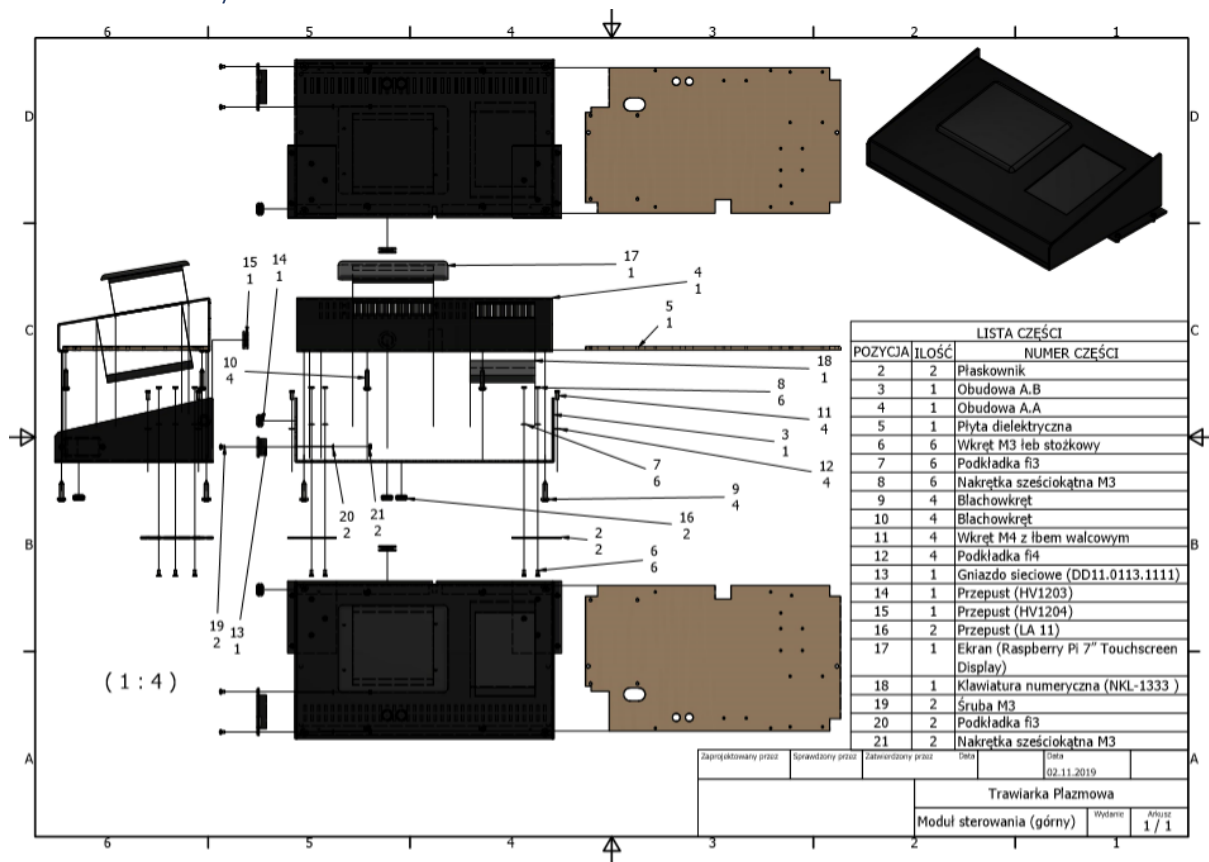
3.2.4. Dolny moduł sterowania



Rys.3.2.5.1 Dolny moduł sterowania – rysunek złożeniowy ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.7.0 Moduł sterowania \(dolny\).pdf](#))

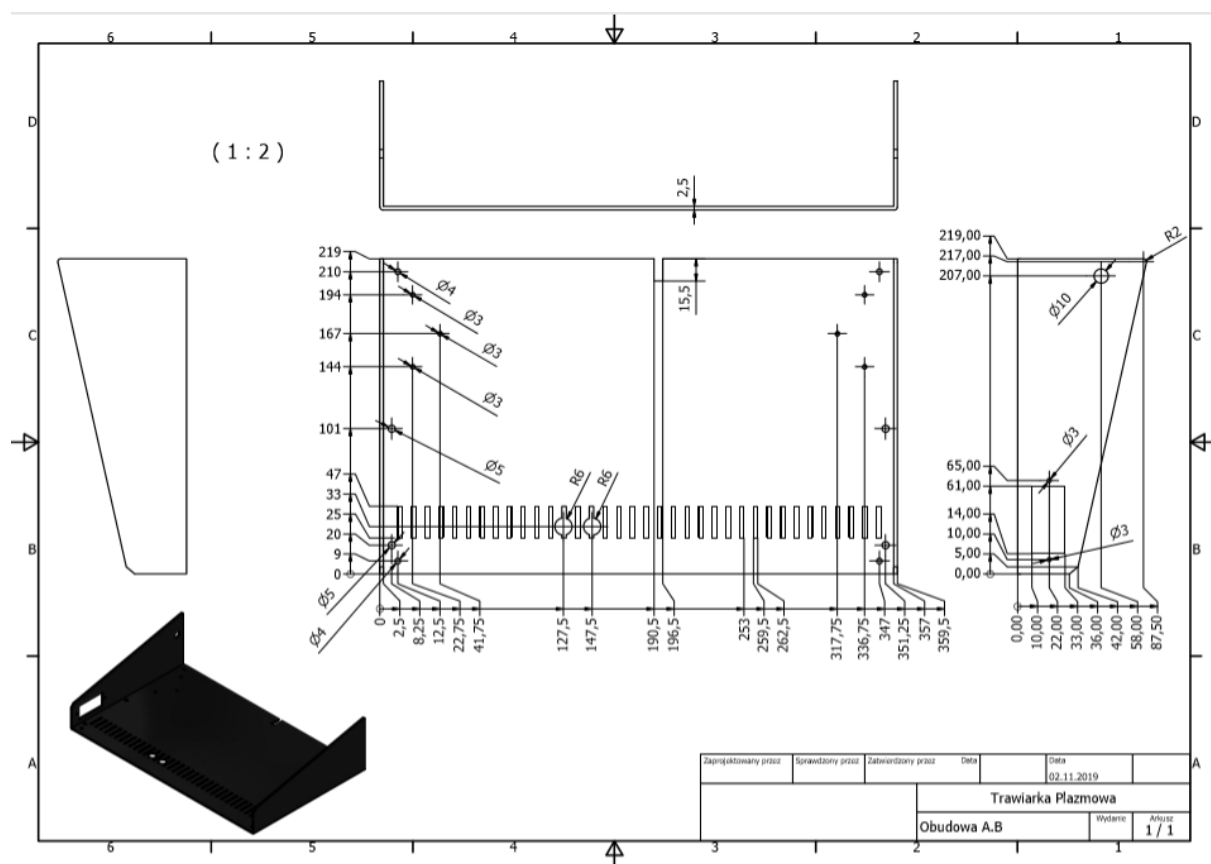
Rys.3.2.5.3 Obudowa B.B ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.7.1 Obudowa B.B.pdf](#))

3.2.5. Górny moduł sterowania

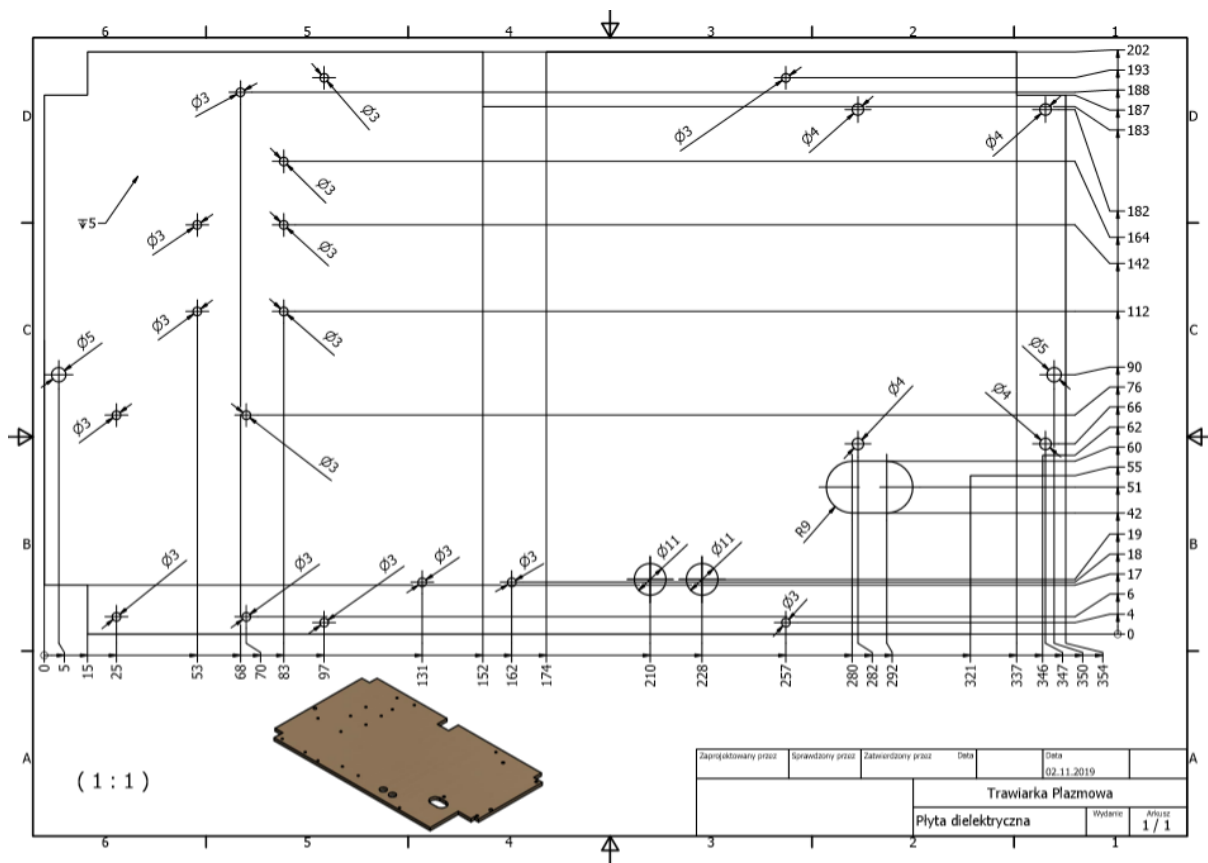


Rys.3.2.6.1 Górny moduł sterowania – rysunek złożeniowy ([Trawiarka Plazowa. Schemat złożeniowy techniczny\1.8.0 Moduł sterowania \(górny\).pdf](#))

Rys.3.2.6.3. Obudowa A.A ([Trawiarka Plazowa. Schemat złozeniowy techniczny\1.8.4 Obudowa A.A.pdf](#))



Rys.3.2.6.4. Obudowa A.B ([Trawiarka Plazowa. Schemat złozeniowy techniczny\1.8.3 Obudowa A.B.pdf](#))



Rys.3.2.6.5. Płyta dielektryczna ([Trawiarka Plazmowa. Schemat złożeniowy techniczny\1.8.5 Płyta dielektryczna.pdf](#))

4. Kod programu

4.1 Wstępne uwagi

- Dane do logowania:

Login: ielplasma

Hasło: 1qaz2wsx3edc

- Dodanie do pliku .bashrc (sudo nano /home/ielplasma/.bashrc) ostatnich trzech linijek powoduje uruchomienie Num Lock na klawiaturze numerycznej oraz uruchomienie się programu wykonawczego urządzenia (_www.py);

```
GNU nano 2.7.4                                     Plik: /home/ielplasma/.bashrc
./etc/bash_completion
fi
fi
export DISPLAY=:0 && /usr/bin/numlockx on
cd scripts
sudo python _www.py
```

Rys.4.1. Zmiana w pliku .bashrc

- Do zrealizowania projektu nie wykorzystano kontroli źródła. Wszelkie zmiany w programie poprzedzone zostały wykonaniem kopii zapasowej i oznakowaniem jej poprzez dodanie kolejnej litery alfabetu przed właściwą nazwą pliku zawierającego skrypt (_www.py);

Serwer zdalny: /home/ielplasma/scripts					
Nazwa pliku	Rozmia...	Typ pliku	Data modyf...	Prawa dostępu	Właściciel/Grupa
..					
data		Folder pl...	03.08.2020 ...	drwxr-xr-x	ielplasma users
instalki		Folder pl...	08.04.2019 ...	drwxr-xr-x	ielplasma users
static		Folder pl...	30.03.2019 ...	drwxr-xr-x	ielplasma users
views		Folder pl...	14.04.2019 ...	drwxr-xr-x	ielplasma users
_www.py	35 537	Python F...	31.07.2020 ...	-rwxr-xr-x	ielplasma users
a_www.py	30 987	Python F...	14.04.2019 ...	-rwxr-xr-x	ielplasma users
b_www.py	31 288	Python F...	14.04.2019 ...	-rwxr-xr-x	ielplasma users
c_www.py	32 781	Python F...	14.04.2019 ...	-rwxr-xr-x	ielplasma users
clear_logs.sh	186	Plik SH	02.04.2019 ...	-rwxr-xr-x	ielplasma users
d_www.py	32 764	Python F...	14.04.2019 ...	-rwxr-xr-x	ielplasma users
e_www.py	32 764	Python F...	17.06.2020 ...	-rwxr-xr-x	ielplasma users
f_www.py	33 118	Python F...	19.06.2020 ...	-rwxr-xr-x	ielplasma users
g_www.py	35 537	Python F...	03.08.2020 ...	-rw-r--r--	ielplasma users
sht71.py	143	Python F...	19.06.2020 ...	-rwxrwxr-x	ielplasma users

Rys.4.2. Katalog scripts – przykład oznaczenia kopii zapasowych.

- Transferu plików dokonano za pomocą programu FileZilla;
- W celach testowych polecenia dla Raspberry Pi wydawano za pomocą programu PuTTY;
- IP urządzenia ustalono za pomocą programu Angry IP Scanner.

4.2 Skrypt wykonawczy

4.2.1. Konfiguracja

```
#!/usr/bin/env python
#link do pomocnej strony: https://bottlepy.org/docs/dev/tutorial_app.html

import sqlite3, datetime, time, serial, threading, os, Queue, spidev, wiringpi
from bottle import request, route, run, template, redirect, get, static_file #post
from sht_sensor import Sht #biblioteka higrometru Sensirion SHT71 (https://pypi.org/project/sht-sensor/)
from subprocess import check_output #na potrzeby sprawdzenia statusu wifi

##### KONFIGURACJA #####
sht71 = Sht(3, 2) #gpio3(pin5) - SCK, gpio 2(pin3) - DATA, piny I2C. Numeracja wg pi3_gpio.png
serial_jrk21v3_dev = "/dev/serial/by-path/platform-3f980000.usb-usb-0:1.2:1.0" #urządzenie wpięte w prawy górny USB
serial_usc02_dev = "/dev/serial/by-path/platform-3f980000.usb-usb-0:1.4:1.0" #urządzenie wpięte w lewy górny USB
database_file = "/home/ielplasma/scripts/data/settings.db"
backup_work_time_file = "/home/ielplasma/scripts/data/backup_work_time.txt" #backup czasu pracy urządzenia
relay_jrk21v3 = 25 #przekaznik wpięty do GPIO 29 wg numeracji wiringpi (pin 40)
relay_usc02 = 29
```



```

relay_gas = 24
#IP_ADDRESS = '172.20.10.2'
IP_ADDRESS = '0.0.0.0'
LOG_FILE = "/home/ielplasma/log/scriptlog.txt"
TMP_LOG_FILE = "/home/ielplasma/log/tmplog.txt" #przechowuje wpisy dot pracy serwomechanizm
ow. Czyszczony przy starcie skryptu
PLATE_START_POS = 2500 #początkowa pozycja ramienia tacy po uruchomieniu
IGN_START_POS = 750*4 #(zakres 750*4 - 2200*4)

MAX_FLOW = 10.0 #maksymalny przepływ -> testuj rozdzielczość

#plytka AD-DA : https://www.waveshare.com/wiki/High-Precision_AD/DA_Board
DAC_CS_PIN = 4 #gpio4 (pin16) DAC8532 chip select, low active
DAC_ch0_diagnostic = 0x30 #adresy kanałów
DAC_ch1_gas_flowmeter = 0x34

ADS_RST_PIN = 1 #gpio1 (pin12) ADS1256 reset input
ADS_CS_PIN = 26 #gpio3 (pin15) ADS1256 chip select, low active
ADS_DRDY_PIN = 0 #gpio0 (pin11) ADS1256 data ready output, low active

#####
#Zmienne Globalne
log_queue = Queue.Queue()
tmp_log_queue = Queue.Queue()
SPI = spidev.SpiDev(0, 0)
##### MAP VALUES #####
# value -> wartość odczytana

# inMin, inMax -> wartości wejściowe (np. zakres litr/min)
# outMin -> outMax -> wartości wyjściowe (np. zakres odczytywanego napięcia)

# return -> zwraca przeskalowaną wartość

def mapValues(value, inMin, inMax, outMin, outMax):
    return (value - inMin) * (outMax - outMin) / (inMax - inMin) + outMin

##### EXAMPLE #####
# flow = mapValues(ADS1256_GetChannelValue(7), Vmin, Vmax, flowMin, flowMax)
# flow = mapValues(3.5, 0, 5, 0, 50)
# -> return (3.5 - 0) * (50 - 0) / (5 - 0) + 0
# -> return (3.5 * 50) / 5
# -> return 175 / 5
# -> return 35
# flow = 35
#####
#####

```

4.2.2. Baza danych

```

def test():
    liczba = datetime.datetime.today().strftime('%S')
    return liczba

def create_connection(db_file):
    """ create a database connection to the SQLite database
        specified by the db_file
    """
    try:
        conn = sqlite3.connect(db_file)
    
```

```

        return conn
    except:
        print_to_log('Problem z podlaczaniem do bazy danych')

    return None

def update_db(value, param):
    """ update bazy danych
    :param = (wartosc, 'nazwa_parametru_do_aktualizacji'):
    :np update_db(5, 'Przeplyw_gazu')
    """

    if(param == 'Przeplyw_gazu' and float(value) > MAX_FLOW):
        value = MAX_FLOW

    conn = create_connection(database_file) #otwieramy polaczenie z baza
    sql = ''' UPDATE settings
              SET value = ?
              WHERE parameter = ?'''
    cur = conn.cursor()
    cur.execute(sql, (value, param,)) #aktualizujemy nowa wartoscia zmienna parametr w tabe
li settings
    conn.commit() #zapisujemy zmiany
    cur.close() #zamykamy polaczenie z baza

def read_db(param):
    """ odczyt z bazy danych
    :param = nazwa_parametru_do_odczytu
    :np read_db('L_przycisk_www')
    """

    conn = create_connection(database_file) #otwieramy polaczenie z baza
    sql = ''' SELECT value
              FROM settings
              WHERE parameter = ?'''

    cur = conn.cursor()
    cur.execute(sql, (param,))
    data = cur.fetchall()
    cur.close() #zamykamy polaczenie z baza
    return data[0][0] #zwraca tylko jedna wartosc

```

4.2.3. Jrk

```

##### sterownik mechanizmu tacy jrk21v3 #####

def jrk21v3_send_data(chr1,chr2):
    #funkcja skleja dane i wysyla po serialu dwa znaki sterujace w kodzie ASCII
    data = chr(chr1) + chr(chr2)
    serial_jrk21v3.write(data)
    serial_jrk21v3.flush()

def jrk21v3_read_data(chr1):
    jrk21v3_send_data(chr1,0)
    read_bytes = serial_jrk21v3.read(2)
    serial_jrk21v3.flushInput()
    if read_bytes == "":
        print_to_log ("!! jrk21v3 nie odpowiedzial - brak komunikacji?? !!")
        print_to_tmp ("!! jrk21v3 nie odpowiedzial - brak komunikacji?? !!")

```

```

        return 0
    else:
        print_to_tmp ("odczytano dane z jrk21v3 ")
        #print_to_tmp (read_bytes)
        return read_bytes

def feedback_plate():
    #zwraca aktualna pozycje z zakresu 0-
4095: str 35 https://www.pololu.com/docs/pdf/0J38/jrk\_motor\_controller.pdf
    feedback = jrk21v3_read_data(0xA7)
    if feedback > 0 :
        arr = list(feedback)
        a = ord(arr[0]) #zamiana pierwszego znaku chr -> byte
        b = ord(arr[1])
        feedb_int = (b << 8) | a
        return feedb_int

def move_plate(position):
    #max wartosc position to 4095 - 12 bitow
    #str 31 https://www.pololu.com/docs/pdf/0J38/jrk\_motor\_controller.pdf
    #print_to_tmp(position)
    dataL = (position & int('00011111',2)) + int('11000000',2) #pobieramy 5 najmlodszych b
itow
    dataH = (position >> 5) & int('01111111',2) #przesuwamy liczbe o 5 poz i pobieramy 7 na
jstarszych bitow

    jrk21v3_send_data(dataL,dataH)

```

4.2.4. DAC8532

```

##### DAC 8532 #####

def DAC8532_Write_Data(Ch, Data):
    global SPI, DAC_CS_PIN
    wiringpi.digitalWrite(DAC_CS_PIN, 0)
    SPI.writebytes([Ch, Data >> 8, Data & 0xff])
    wiringpi.digitalWrite(DAC_CS_PIN, 1)

def DAC8532_Out_Voltage(kanal, Voltage):
    DAC_VREF = 5
    DAC_Value_MAX = 65535
    if((Voltage <= DAC_VREF) and (Voltage >= 0)):
        temp = int(Voltage * DAC_Value_MAX / DAC_VREF)
        DAC8532_Write_Data(kanal, temp)
        print_to_log('    Napiecie wyjsciowe = {}V'.format(Voltage))
        print_to_log('    Po przekształceniu = {}'.format(temp))

def DAC8532_Out_Flow(flow):
    # mapValues(flow, flowMin, flowMax, VoltageMin, VoltageMax)
    DAC8532_Out_Voltage(DAC_ch1_gas_flowmeter, mapValues(flow, 0, 10, 0, 5))

```

4.2.5. ADS1256

```

##### ADS 1256 #####

def ADS1256_Read_data(reg):
    global SPI, ADS_CS_PIN
    wiringpi.digitalWrite(ADS_CS_PIN, 0)
    SPI.writebytes([0x10 | reg, 0x00])
    data = SPI.readbytes(1)

```

```

wiringpi.digitalWrite(ADS_CS_PIN, 1)
return data

def ADS1256_Write_data(data):
    global SPI, ADS_CS_PIN
    wiringpi.digitalWrite(ADS_CS_PIN, 0)
    SPI.writebytes([data])
    wiringpi.digitalWrite(ADS_CS_PIN, 1)

def ADS1256_Write_reg(reg, data):
    global SPI, ADS_CS_PIN
    wiringpi.digitalWrite(ADS_CS_PIN, 0)
    SPI.writebytes([0x50 | reg, 0x00, data]) #'CMD_WREG' : 0x50
    wiringpi.digitalWrite(ADS_CS_PIN, 1)

def ADS1256_WaitDRDY():
    global ADS_DRDY_PIN
    for i in range(0,40,1):
        if(wiringpi.digitalRead(ADS_DRDY_PIN) == 0):
            break
    if(i == 39):
        print_to_log('!! Timeout przy oczekiwaniu na odpowiedz z ADS1256 (plytka AD-DA)')
        print ('!! Timeout przy oczekiwaniu na odpowiedz z ADS1256 (plytka AD-DA)')
        time.sleep(0.1)

def ADS1256_Read_ADC_Data():
    global SPI, ADS_CS_PIN
    wiringpi.digitalWrite(ADS_CS_PIN, 0)
    SPI.writebytes([0x01]) #'CMD_RDATA' : 0x01
    time.sleep(0.01)
    buf = SPI.readbytes(3)
    wiringpi.digitalWrite(ADS_CS_PIN, 1)
    read = (buf[0]<<16) & 0xff0000
    read |= (buf[1]<<8) & 0xff00
    read |= (buf[2]) & 0xff
    if (read & 0x800000):
        read |= 0xFF000000
    return read

def ADS1256_GetChannelValue(Channel):
    global ADS_DRDY_PIN
    if read_db('ADS1256') == 'OK':
        while(wiringpi.digitalRead(ADS_DRDY_PIN) == 1):
            pass
        if(Channel>=8):
            return 0
        elif(Channel == 7):
            Channel = 0
        else :
            Channel = Channel + 1

    ADS1256_Write_reg(1, (Channel<<4) | (1<<3))
    ADS1256_Write_data(0xFC) #'CMD_SYNC' : 0xFC
    time.sleep(0.01)
    ADS1256_Write_data(0x00) #'CMD_WAKEUP' : 0x00
    time.sleep(0.01)
    val = ADS1256_Read_ADC_Data()
    #Value = val*5.0/0x7fffff

```

```

        #Value_short = '%.2f'%(Value) #zostawia tylko 2 liczby po przecinku
        #return Value
        return val
    else:
        return 'x'

def ADS1256_GetAll():
    ADC_Value = [0,0,0,0,0,0,0,0]
    for i in range(0,8,1):
        ADC_Value[i] = '%.2f'%(ADS1256_GetChannelValue(i)*5.0/0x7fffff)
        if i == 7:
            print_to_log(ADS1256_GetChannelValue(i))
    return ADC_Value

def ADS1256_Init():
    #Konfiguracja ADS1256 podlaczzonego po SPI: https://www.waveshare.com/wiki/High-Precision\_AD/DA\_Board
    global SPI, DAC_CS_PIN, ADS_RST_PIN, ADS_CS_PIN, ADS_DRDY_PIN

    wiringpi.pinMode(ADS_RST_PIN, wiringpi.OUTPUT)
    wiringpi.pinMode(ADS_CS_PIN, wiringpi.OUTPUT)
    wiringpi.pinMode(ADS_DRDY_PIN, wiringpi.INPUT)
    wiringpi.pullUpDnControl(ADS_DRDY_PIN, wiringpi.PUD_UP) #pullup na tym wejsci

    SPI.max_speed_hz = 20000
    SPI.mode = 0b01

    #HW reset modulu ADS1256
    wiringpi.digitalWrite(ADS_RST_PIN, 1)
    time.sleep(0.2)
    wiringpi.digitalWrite(ADS_RST_PIN, 0)
    time.sleep(0.2)
    wiringpi.digitalWrite(ADS_RST_PIN, 1)

    ADS1256_WaitDRDY() #czekamy az odpowie

    id = ADS1256_Read_data(0)
    id = id[0] >> 4

    if id == 3 :
        ADS1256_WaitDRDY()
        buf = [0,0,0,0,0,0,0,0]
        buf[0] = (0<<3) | (1<<2) | (0<<1)
        buf[1] = 0x08
        buf[2] = (0<<5) | (0<<3) | (0<<0) #(0<<5) | (0<<3) | (gain<<0) -> ADS1256_GAIN_1 = 0
        buf[3] = 0x33 #ADS1256_15SPS

        wiringpi.digitalWrite(ADS_CS_PIN, 0)
        SPI.writebytes([0x50 | 0, 0x03])
        SPI.writebytes(buf)
        wiringpi.digitalWrite(ADS_CS_PIN, 1)
        update_db('OK', 'ADS1256')
        time.sleep(0.001)
    else:
        print('!!! Problem z ukladem ADS1256, podaje bledne id={} a powinno byc id=3 (plytka A
D-DA)'.format(id))

```

```

        print_to_log('!! Problem z układem ADS1256, podaje bledne id={} a powinno byc id=3 (p
lytka AD-DA)'.format(id))
        update_db('!! Nie wykryto !!', 'ADS1256')
        update_db('1', 'SELFTEST_ERROR')

```

4.2.6. usc02a

```

##### sterownik zaplonu plazmy usc02a #####

def ign_servo_send_data(chr1,chr2,chr3,chr4):
    #funkcja skleja dane i wysyla po serialu dwa znaki sterujace w kodzie ASCII
    data = chr(chr1) + chr(chr2) + chr(chr3) + chr(chr4)
    serial_usc02.write(data)
    serial_usc02.flush()
    print_to_tmp ("Wyslano dane do sterownika USC02")

def move_ign_servo(position):
    dataL = (position & int('01111111',2)) #pobieramy 7 najmlodszych bitow
    dataH = (position >> 7) & int('01111111',2) #przesuwamy liczbe o 5 poz i pobieramy 7 na
jstarszych bitow
    #print_to_tmp('wysylam dataL: ' + str(dataL) + ',dataH: ' + str(dataH))
    ign_servo_send_data(0x84, 0x00, dataL, dataH)

```

4.2.7. Wątki

```

##### THREADS #####

def SHT_temp_measure():

    while read_db('SHUTDOWN') != 1: #zmienna shutdown przyjmuje wartosc 1 gdy recznie chcemy
zamknac skrypt (CTRL-C)
        try:
            temp_sht71 = int(sht71.read_t())
            rh_sht71 = int(sht71.read_rh())
            update_db(temp_sht71, 'Temp_SHT71')
            update_db(rh_sht71, 'RH_SHT71')
            update_db('OK', 'SHT71')

        except:
            print_to_log('Problem z odczytem temp z czujnika SHT')
            update_db('?', 'Temp_SHT71')
            update_db('?', 'RH_SHT71')
            update_db('!!! Utracono komunikacje !!!', 'SHT71')

    time.sleep(3)

    print_to_log('SHT Thread - wykryto sygnał zamknięcia')

def work_time_measurement():

    while read_db('SHUTDOWN') != 1: # zmienna shutdown przyjmuje wartosc 1 gdy recznie chcemy
zamknac skrypt (CTRL-C)
        for heartbeat_counter in range(30):
            if read_db('PROCESS_STATE') == 'running':
                #print('poczatek pomiaru czasu pracy')
                while read_db('PROCESS_STATE') == 'running':
                    work_time += 1
                    time.sleep(1)

            total_work_time = read_db('Czas_pracy_urz')

```

```

        total_work_time = total_work_time + work_time
        update_db(total_work_time, 'Czas_pracy_urz')
        #zapis backupowego czasu pracy rowniez do pliku
        backup_time_file = open(backup_work_time_file, "r") #backupowy plik przechowujac
y czas
        backup_time= int(backup_time_file.read()) + work_time
        backup_time_file = open(backup_work_time_file, "w")
        backup_time_file.write(str(backup_time))
        backup_time_file.close()
        #print('koniec pomiaru czau pracy')
    else:
        work_time = 0

    time.sleep(0.1)
    print_to_log('Work Time Measurement Thread - zakonczył prace')

def print_to_log_queue():
    while True:
        text = log_queue.get()
        log = open(LOG_FILE, "r")
        data = datetime.datetime.today().strftime('%d.%m.%Y %H:%M:%S')
        firstline = ('{:}: {}'.format(data, text))
        oline=log.readlines()
        oline.insert(0,firstline)
        log.close()
        log=open(LOG_FILE,"w")
        log.writelines(oline)
        log.close()
        log_queue.task_done()

def print_to_tmp_log_queue():
    while True:
        text = tmp_log_queue.get()
        tmplog = open(TMP_LOG_FILE, "a")
        data = datetime.datetime.today().strftime('%d.%m.%Y %H:%M:%S')
        tmplog.write('{}: {}'.format(data, text))
        tmplog.close()
        tmp_log_queue.task_done()

```

4.2.8. Main

```

def main_process():

    job_aborted=0

    while read_db('SHUTDOWN') != 1: #zmienna shutdown przyjmuje wartosc 1 gdy recznie chcemy
zamknac skrypt (CTRL-C)
        for heartbeat_counter in range(30):
            if read_db('L_przycisk_www') == 'Zatrzymaj Przebieg':
                speed = read_db('Predkosc_przesuwu')
                iteration_cnt = read_db('Ilosc_iteracji')
                act_position = feedback_plate()
                start_position = 2700
                mid_position = 900
                end_position = 180
                ignition_pos = 1000 #poz przy ktorej ma nastapic zaplon plazmy

```

```

    ignition_process_start()
    check_plate_state(start_position) #przesuwamy tace na poz startowa jesli sie tam ni
e znajduje

    for iteration in range(iteration_cnt):
        update_db('running', 'PROCESS_STATE')
        print_to_log(' Iteracja {} z {}'.format(iteration+1,iteration_cnt))
        update_db('Iteracja {} z {}'.format(iteration+1,iteration_cnt),'Status')
        act_position = feedback_plate()
        print_to_tmp('act_position: {}, end_position: {}'.format(act_position, end_positi
on))

        new_position = act_position
        while act_position > (end_position + 50) and read_db('L_przycisk_www') == 'Zatrzy
maj Przebieg':
            new_position = new_position - speed
            if new_position > (end_position + 30):
                move_plate(new_position)
                print_to_tmp('new_position: ' + str(new_position))
            else:
                move_plate(end_position)
            time.sleep(.100)
            act_position = feedback_plate() #odswierzamy pozycje
            print_to_tmp('act_position: ' + str(act_position))

        if iteration + 1 != iteration_cnt: #+1 bo iteracja startuje od 0
            target_position = mid_position #jesli nie jest to ostatni przebieg to wroc na
poz mid
        else:
            target_position = start_position #przy ostatnim przebiegu wroc na poz startowa

        new_position = act_position
        while act_position < (target_position - 50) and read_db('L_przycisk_www') == 'Zat
rzymaj Przebieg':
            new_position = new_position + speed
            if new_position < (target_position - 30):
                move_plate(new_position)
                print_to_tmp('new_position: ' + str(new_position))
            else:
                move_plate(target_position)
            time.sleep(.100)
            act_position = feedback_plate() #odswierzamy pozycje
            print_to_tmp('act_position: ' + str(act_position))

        if read_db('L_przycisk_www') == 'Uruchom Przebieg':
            print_to_log('>> Wstrzymano przebieg')
            print_to_tmp('>> Wstrzymano przebieg')
            update_db('standby', 'PROCESS_STATE')
            job_aborted = 1
            break

    update_db('Uruchom Przebieg', 'L_przycisk_www')
    update_db('Oczekiwanie', 'Status')

    if job_aborted == 0:
        update_db('standby', 'PROCESS_STATE')
        print_to_log('>> Zakonczo przebieg')

```



```

        print_to_log(' ')
        print_to_tmp('>> Zakonczono przebieg')
    else:
        job_aborted = 0 #resetujemy stan licznika

    time.sleep(0.1)
##### Funkcje dot main_process #####

def ignition_process_start():
    servo_ign_pos_min = 750*4 #min pozycja
    servo_ign_pos_max = 2200*4

    flow = read_db('Przeplyw_gazu')
    #DAC8532_Out_Flow(flow)
    voltage = mapValues(flow, 0.5, MAX_FLOW, 0.0, 5.0)
    DAC8532_Out_Voltage(DAC_ch1_gas_flowmeter, voltage)
    print_to_log(' Ustalam przeplyw gazu: {}l/min'.format(flow))
    ADC_Value = ADS1256_GetAll()
    print_to_log(' Odczytane napiecie: {}V'.format(ADC_Value[7]))

    print_to_log(' Zaplon')
    print_to_tmp(' Zaplon')

    move_ign_servo(servo_ign_pos_max)
    time.sleep(0.5)
    move_ign_servo(servo_ign_pos_min)

def check_plate_state(position):
    act_position = feedback_plate()
    if (position - 150) > act_position or act_position > (position + 150): #sprawdzamy i ew u
stawiamy tace na poz startowej
        print_to_log(' Taca nie znajduje sie na poz startowej. Przesuwanie tacy')
        print_to_tmp(' Taca nie znajduje sie na poz startowej. Przesuwanie tacy')
        move_plate(position)
        while (position - 100) > act_position or act_position > (position + 100):
            time.sleep(1)
            act_position = feedback_plate()
            print_to_tmp('.'),
        print_to_log(' Gotowe! Taca znajduje sie na pozycji startowej')
        print_to_tmp(' Gotowe! Taca znajduje sie na pozycji startowej')

#####

def print_to_log(text):
    """Funkcja wysyla tekst do kolejki.
    Obsluga kolejki zajmuje sie watek: GS_print_to_log_thread,
    ktory wysyla dane do funkcji print_to_log_queue()
    """
    log_queue.put(text)

def print_to_tmp(text):
    tmp_log_queue.put(text)

def selftest():

```

```

try:
    #inicjalizacja UARTA dla przesuwu tacy (serial_jrk21v3_dev jest usawiany w konfiguracji
na poczatku skryptu
    global serial_jrk21v3 #definiowanie zmiennej globalnej
    serial_jrk21v3 = serial.Serial(serial_jrk21v3_dev, baudrate=9600, parity=serial.PARITY_
NONE, stopbits=serial.STOPBITS_ONE,
    bytesize=serial.EIGHTBITS, writeTimeout = 0, timeout = 5, rtscts=False, dsrdtr=False, x
onxoff=False)
except:
    update_db('1','SELFTEST_ERROR')
    update_db('!! Nie wykryto !!','JRK21V3')
    print_to_log("!! Problem z inicjalizacja sterownika jrk21v3. Czy jest w porcie USB?")
    print_to_tmp("!! Problem z inicjalizacja sterownika jrk21v3. Czy jest w porcie USB?")
    print("!! Problem z inicjalizacja sterownika jrk21v3. Czy jest w porcie USB? Czy skrypt
ma uprawnienia do portow USB?")

try:
    global serial_usc02 #definiowanie zmiennej globalnej
    serial_usc02 = serial.Serial(serial_usc02_dev, baudrate=9600, parity=serial.PARITY_NONE
, stopbits=serial.STOPBITS_ONE,
    bytesize=serial.EIGHTBITS, writeTimeout = 0, timeout = 5, rtscts=False, dsrdtr=False, x
onxoff=False)
except:
    update_db('1','SELFTEST_ERROR')
    update_db('!! Nie wykryto !!','USC02')
    print_to_log('!! Problem z inicjalizacja sterownika zaplonu USC02. Czy jest w porcie US
B?')
    print_to_tmp('!! Problem z inicjalizacja sterownika zaplonu USC02. Czy jest w porcie US
B?')
    print('!! Problem z inicjalizacja sterownika zaplonu USC02. Czy jest w porcie USB? Czy
skrypt ma uprawnienia do portow USB?')

try:
    temp_sht71 = int(sht71.read_t())
except:
    print_to_log('!! Problem z odczytem temp z czujnika SHT71')
    print('!! Problem z odczytem temp z czujnika SHT71')
    update_db('!! Nie wykryto !!','SHT71')
    update_db('1','SELFTEST_ERROR')

ADS1256_Init()
wiringpi.pinMode(DAC_CS_PIN, wiringpi.OUTPUT)

```

4.2.9. Bottle (web framework)

```

##### BOTTLE #####

L_przycisk_www = read_db('L_przycisk_www')

@route('/diagnostic')
def diagnostic():

    dane = {
        'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie dd
.mm.yyyy
        'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:mm
        'status' : read_db('Status'), #odczyt statusu z bazy danych
        'refresh_www_time' : 5, #odswierzanie strony w sekundach
    }

```

```

        'JRK21V3' : read_db('JRK21V3'),
        'USC02' : read_db('USC02'),
        'SHT71' : read_db('SHT71'),
        'ADS1256' : read_db('ADS1256'),
    }

    if read_db('JRK21V3') == 'OK' and read_db('USC02') == 'OK': return template ('diagnosti
c_all.tpl', dane)
    if read_db('JRK21V3') == 'OK' and read_db('USC02') != 'OK': return template ('diagnosti
c_jrk.tpl', dane)
    if read_db('JRK21V3') != 'OK' and read_db('USC02') == 'OK': return template ('diagnosti
c_usc.tpl', dane)
    if read_db('JRK21V3') != 'OK' and read_db('USC02') != 'OK': return template ('diagnosti
c_none.tpl', dane)

    return template ('diagnostic.tpl', dane)

@route('/favicon.ico')
def get_favicon():
    return static_file('favicon.ico', root='/home/ielplasma/scripts/static')

@route('/<filename:path>')
def download(filename):
    return static_file(filename, root='/home/ielplasma/log/', mimetype='text/plain') #download
=filename) #mimetype='text/plain')

@route('/settings')
def settings():

    #Zmienne przesyłane do strony www poprzez template main.tpl
    dane = {
        'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie dd
.mm.yyyy
        'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:mm
        'temp1' : read_db('Temp_SHT71'), #temp_sht71,
        'temp2' : test(), #"25",
        'rh' : read_db('RH_SHT71'), #rh_sht71,
        'predkosc': read_db('Predkosc_przesuwu'),
        'moc': read_db('Moc_plazmy'),
        'przeplyw' : read_db('Przeplyw_gazu'),
        'ilosc' : read_db('Ilosc_iteracji'),
        'cz_oczek' : read_db('Czas_oczekiwania'),
    }

    return template ('settings.tpl', dane)

@route('/start')
def start():

    if read_db('L_przycisk_www') == 'Uruchom Przebieg' and read_db('Status') != '! BLAD !'
:
        update_db('Zatrzymaj Przebieg', 'L_przycisk_www')
        print_to_log("<< Uruchomiono przebieg")
        print_to_tmp("<< Uruchomiono przebieg")

    elif read_db('L_przycisk_www') == 'Zatrzymaj Przebieg' :

```

```

        update_db('Uruchom Przebieg', 'L_przycisk_www')

    elif read_db('Status') == '! BLAD !' :
        dane = {
            'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie dd
dd.mm.yyyy
            'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:
mm
            'status' : read_db('Status'), #odczyt statusu z bazy danych
            'refresh_www_time' : 15,
        }
        return template ('blad.tpl', dane)
    redirect("/")

@route('/shutdown')
def shutdown_ask():

    dane = {
        'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie dd
.mm.yyyy
        'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:mm
        'temp1' : read_db('Temp_SHT71'),
        'temp2' : test(), #"25",
        'rh' : read_db('RH_SHT71'),
        'status' : read_db('Status'), #odczyt statusu z bazy danych
        'refresh_www_time' : 5,
    }

    return template ('shutdown.tpl', dane)

@route('/shutdown_yes')
def shutdown_yes():

    safe_temp = 30
    temperatura = read_db('Temp_SHT71')
    if temperatura > safe_temp and read_db('SHT71') == "OK":
        dane = {
            'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie
dd.mm.yyyy
            'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:
mm
            'temp1' : temperatura,
            'temp2' : test(), #"25",
            'rh' : read_db('RH_SHT71'),
            'refresh_www_time' : 3,
            'safe_temp' : safe_temp,
        }
        return template ('shutdown_overtemp.tpl', dane)

    else:
        bashCommand = "sudo init 0"
        os.system(bashCommand)

@route('/test_plate_servo')
def test_plate_servo():

    if read_db('Status') == 'Oczekiwanie' or read_db('Status') == '! BLAD !':

```

```

        print_to_log('Uruchomiono test serwo przesuwu probki')
        print_to_tmp('Uruchomiono test serwo przesuwu probki')
        move_plate(PLATE_START_POS - 500)
        while feedback_plate() > PLATE_START_POS - 400:
            time.sleep(0.2)
        move_plate(PLATE_START_POS)

    redirect("/diagnostic")

@route('/test_ign_servo')
def test_ign_servo():

    if read_db('Status') == 'Oczekiwanie' or read_db('Status') == '! BLAD !':
        print_to_log('Uruchomiono test serwo zaplonu')
        print_to_tmp('Uruchomiono test serwo zaplonu')
        move_ign_servo(2200*4)
        time.sleep(0.5)
        move_ign_servo(IGN_START_POS)

    redirect("/diagnostic")

@route('/test_ad_da_board')
def test_ad_da_board():

    #podlaczenia ADS1256: AD0: potencjometr, AD1: cz_swiatla, AD2: DAC ch0, AD3: GND, AD5:
+5V, AD7: gas flowmeter
    ADC_Value = ADS1256_GetAll()

    dane = {
        'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie dd
.mm.yyyy
        'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:mm
        'temp1' : read_db('Temp_SHT71'),
        'temp2' : test(), #"25",
        'rh' : read_db('RH_SHT71'),
        'status' : read_db('Status'), #odczyt statusu z bazy danych
        'refresh_www_time' : 3,
        'ADS1256' : read_db('ADS1256'),
        'AD2_DAC0' : ADC_Value[2], #ADS1256_GetChannelValue(2),
        'AD3_GND' : ADC_Value[3],
        'AD5_5V' : ADC_Value[5],
        'AD7_gas' : ADC_Value[7],
        'diag_voltage' : 2.5,
    }

    return template ('test_ad_da_board.tpl', dane)

@route('/')
def index():
    change_settings_var = 0

    L_przycisk_www = read_db('L_przycisk_www') #na potrzeby zmiennej globalnej

    predkosc = request.query.get('predkosc') #odczytujemy z paska adresu zmienna predkosc
    if predkosc == None or predkosc == "" : predkosc = read_db('Predkosc_przesuwu') #jesli
jest pusta to podstawiamy wartosc z bazy danych

```

```

    else: update_db(predkosc, 'Predkosc_przesuwu'); change_settings_var += 1 #jesli nie je
st pusta to aktualizujemy baze danych

moc = request.query.get('moc')
if moc == None or moc == "" : moc = read_db('Moc_plazmy')
else: update_db(moc, 'Moc_plazmy'); change_settings_var += 1

przeplyw = request.query.get('przeplyw')
if przeplyw == None or przeplyw == "" : przeplyw = read_db('Przeplyw_gazu')
else: update_db(przeplyw, 'Przeplyw_gazu'); change_settings_var += 1

ilosc = request.query.get('ilosc')
if ilosc == None or ilosc == "" : ilosc = read_db('Ilosc_iteracji')
else: update_db(ilosc, 'Ilosc_iteracji'); change_settings_var += 1

cz_oczek = request.query.get('cz_oczek')
if cz_oczek == None or cz_oczek == "" : cz_oczek = read_db('Czas_oczekiwania')
else: update_db(cz_oczek, 'Czas_oczekiwania'); change_settings_var += 1

#gdz zostala zmieniona choc jedna zmienna to przeladuj strone aby w pasku adresu zostal
tylko adres bez zmiennych
if change_settings_var > 0: redirect(request.path)

#ADC_Value = ADS1256_GetAll()
przeplyw_aktualny = 55 #ADC_Value[7]

total_work_time_d_string = time.strftime('%j', time.gmtime(read_db('Czas_pracy_urz')))
total_work_time_d = int(total_work_time_d_string)-
1 #domyslnie biblioteka time podaje wartosc 1-366, wiec odejmujemy 1 aby wskazywalo 0
total_work_time_h = int(time.strftime('%H', time.gmtime(read_db('Czas_pracy_urz'))))
total_work_time_m = int(time.strftime('%M', time.gmtime(read_db('Czas_pracy_urz'))))
total_work_time = str(total_work_time_d)+'d, '+str(total_work_time_h)+'h '+str(total_wo
rk_time_m)+'m '+time.strftime(' %Ss', time.gmtime(read_db('Czas_pracy_urz'))))

if read_db('L_przycisk_www') == "Zatrzymaj Przebieg":
    refresh_www_time = 1
else:
    refresh_www_time = 5

#Zmienne przesylane do strony www poprzez template main.tpl
dane = {
    'akt_data': datetime.datetime.today().strftime('%d.%m.%Y'), #data w formacie dd
.mm.yyyy
    'godzina' : datetime.datetime.today().strftime('%H:%M'), #godz w formacie hh:mm
    'temp1' : read_db('Temp_SHT71'), #temp_sht71,
    'temp2' : test(), #"25",
    'rh' : read_db('RH_SHT71'), #rh_sht71,
    'status' : read_db('Status'), #odczyt statusu z bazy danych
    'moc_aktualna' : "150",
    'przeplyw_aktualny' : przeplyw_aktualny, #"15",
    'predkosc': predkosc,
    'moc': moc,
    'przeplyw' : przeplyw,
    'ilosc' : ilosc,
    'cz_oczek' : cz_oczek,
    'cz_pracy_urz' : total_work_time, #"130h",
    'zuzycie_gazu' : "52",
    'lewy_przycisk' : read_db('L_przycisk_www'), #L_przycisk_www,

```

```

        'refresh_www_time' : refresh_www_time,
    }

    return template ('main.tpl', dane)

```

4.2.10. Część wykonawcza

```

##### POZATEK PROGRAMU #####

print_to_log(' ')
print_to_log('-----')

#resetujemy niektóre wpisy bazy danych do ust fabrycznych
update_db('Uruchom Przebieg', 'L_przycisk_www') #przy starcie aplikacji ustawiamy defaultowe nazwy
update_db('Oczekiwanie', 'Status')
update_db('x', 'Temp_SHT71')
update_db('x', 'RH_SHT71')
update_db('0', 'SELFTEST_ERROR') #na potrzeby zgłaszania problemów ze sprzętem przy starcie
update_db('OK', 'JRK21V3')
update_db('OK', 'USC02')
update_db('OK', 'SHT71')
update_db('x', 'ADS1256')
update_db('0', 'SHUTDOWN')
update_db('standby', 'PROCESS_STATE')

os.system('echo -n > {}'.format(TMP_LOG_FILE)) #czyszcimy plik z tymczasowymi logami

if read_db('Czas_pracy_urz') == 0: #jesli przy starcie okaze sie, ze w bazie nie ma calkowiciego czasu pracy
    backup_time_file = open(backup_work_time_file, 'r') #backupowy plik przechowujacy calkowity czas pracy
    update_db(int(backup_time_file.read()), 'Czas_pracy_urz')
    print_to_log('!! Brak calkowitego czasu pracy w bazie danych, przywrócono czas z pliku backupu!')
    print('!! Brak calkowitego czasu pracy w bazie danych, przywrócono czas z pliku backupu!')
)

GS_SHT_temp_thread = threading.Thread(target=SHT_temp_measure) #definiujemy watek do pracy w tle (uruchamiamy go w innym miejscu)
GS_SHT_temp_thread.setName('!! GS_SHT_temp_thread !!') #podczas bledu wyswietli nazwe watku i latwo bedzie debugowac
GS_main_process_thread = threading.Thread(target=main_process)
GS_main_process_thread.setName('!! GS_main_process_thread !!')
GS_work_time_measurement_thread = threading.Thread(target=work_time_measurement)
GS_work_time_measurement_thread.setName('!! GS_work_time_measurement_thread !!')

GS_print_to_log_thread = threading.Thread(target=print_to_log_queue)
GS_print_to_log_thread.setName('!! GS_print_to_log_thread !!')
GS_print_to_log_thread.daemon = True
GS_print_to_log_thread.start()

GS_print_to_tmp_log_thread = threading.Thread(target=print_to_tmp_log_queue)
GS_print_to_tmp_log_thread.setName('!! GS_print_to_tmp_log_thread !!')
GS_print_to_tmp_log_thread.daemon = True
GS_print_to_tmp_log_thread.start()

```

```

wiringpi.wiringPiSetup() #inicjalizacja wiringpi

selftest() #funkcja sprawdza czy rpi widzi cz temp sht71 i sterowniki serwo jrk21v3 oraz us
c02

if read_db('SELFTEST_ERROR') == 0: #gdy selftest nie zwraca bledow to
    move_plate(PLATE_START_POS)
    move_ign_servo(IGN_START_POS)

    #wlaczenie przekaznikow
    wiringpi.digitalWrite(relay_jrk21v3, 0) #przekazniki wlaczone stanem niskim na porcie
    wiringpi.pinMode(relay_jrk21v3, 1) #ustawienie przekaznika w tryb wyjścia

    wiringpi.digitalWrite(relay_usc02, 0)
    wiringpi.pinMode(relay_usc02, 1)

    wiringpi.digitalWrite(relay_gas, 0)
    wiringpi.pinMode(relay_gas, 1)

    #DAC_ testowo ustawiamy napiecia na kanalach
    DAC8532_Out_Voltage(DAC_ch0_diagnostic, 2.5)

    # uruchamiamy watki
    GS_work_time_measurement_thread.start()
    GS_main_process_thread.start()

else:
    update_db('! BLAD !', 'Status')

if read_db('SHT71') == "OK": #jesli po selftescie wykryto czujnik sht71 to wystartuj watek
pomiaru temp
    GS_SHT_temp_thread.start() #uruchamiamy watek
    #print_to_log("Uruchomiono watek pomiaru temperatury z czujnika SHT71")
    #print_to_log("Wylaczono uruchamie watka pomiaru temperatury z czujnika SHT")

print_to_log('***** Uruchamianie Bottle *****')
print_to_log(' ')
run(host=IP_ADDRESS, port=8080, debug=False, reloader=False)

#while True:
#    eth_ip = check_output(['hostname', '-I']).strip()
#    if eth_ip == IP_ADDRESS:
#        print_to_log('***** Uruchamianie Bottle *****')
#        print_to_log(' ')
#        run(host=IP_ADDRESS, port=8080, debug=False, reloader=False)
#        break
#    else:
#        print_to_log('!! Brak poprawnego adresu IP')
#        time.sleep(5)

```

4.2.11. Wyłączanie programu

```

##### Ponizsze linie wykonaja sie po wcisnieciu CTRL-C na konsoli #####

print_to_log('Wykryto wcisniecie CTRL-C, zamykanie skryptu')

```



```

print('Wykryto wcisniecie CTRL-C, zamykanie skryptu')
update_db(1,'SHUTDOWN') #informuje watki o planowanym zamknieniu

if GS_main_process_thread.isAlive() is True:
    print ("Oczekiwanie na zamkniecie watku 'main_process_thread' (!) ")
if GS_work_time_measurement_thread.isAlive() is True:
    print ("Oczekiwanie na zamkniecie watku 'work_time_measurement_thread' ($)")
if GS_SHT_temp_thread.isAlive() is True:
    print ("Oczekiwanie na zamkniecie watku pomiary temperatury z czujnika SHT (%)")

while GS_main_process_thread.isAlive() is True or GS_work_time_measurement_thread.isAlive()
is True or GS_SHT_temp_thread.isAlive is True:
    if GS_main_process_thread.isAlive() is True:
        print (" ! ")
    if GS_work_time_measurement_thread.isAlive() is True:
        print (" $ ")
    if GS_SHT_temp_thread.isAlive() is True:
        print (" % ")
    time.sleep(1)

if read_db('SELFTTEST_ERROR') == 0: #jesli podczas startu selftest przeszedl poprawnie to
wiringpi.pinMode(relay_jrk21v3, 0) #ustawienie portu w tryb wejscia (default)
move_plate(PLATE_START_POS)
move_ign_servo(IGN_START_POS)

```

4.3 Skrypty pomocnicze

4.3.1. SHT75 (sht71.py)

```

#!/usr/bin/env python

from sht_sensor import Sht

sht = Sht(3, 2)
print 'Temperature', sht.read_t()
print 'Relative Humidity', sht.read_rh()

```

4.3.2. Tworzenie bazy danych (create_database.py)

```

import sqlite3
conn = sqlite3.connect('settings.db') # Warning: This file is created in the current direct
ory
conn.execute("CREATE TABLE settings (id INTEGER PRIMARY KEY, parameter char(100) NOT NULL,
value int NOT NULL)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Predkosc_przesuwu',50)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Moc_plazmy',100)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Przeplyw_gazu',15)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Ilosc_iteracji',1)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Czas_oczekiwania',25)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('L_przycisk_www','Uruchom Prze
bieg')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Status','Oczekiwanie')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Czas_pracy_urz',0)")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('Temp_SHT71','x')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('RH_SHT71','x')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('SELFTTEST_ERROR','0')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('JRK21V3','x')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('USC02','x')")

```

```
conn.execute("INSERT INTO settings (parameter,value) VALUES ('SHT71','x')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('ADS1256','x')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('SHUTDOWN','0')")
conn.execute("INSERT INTO settings (parameter,value) VALUES ('PROCESS_STATE','standby')")
conn.commit()
```